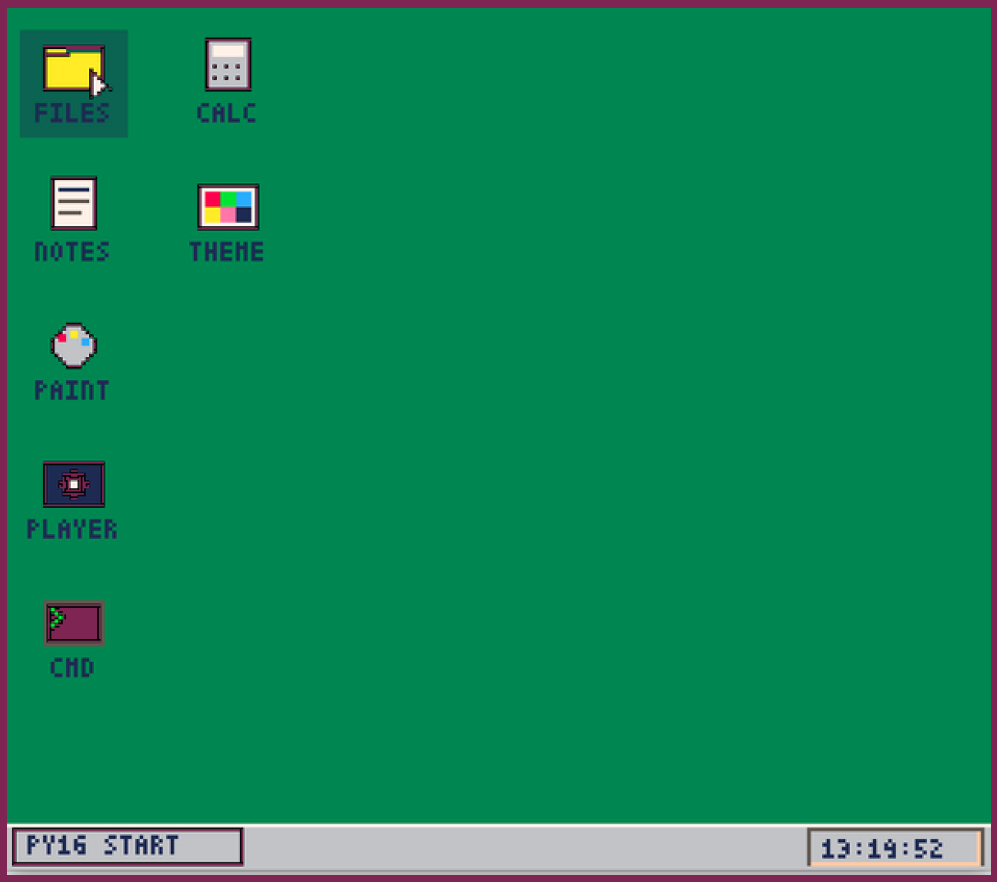
PY16OS CART

PY-16 FANTASY CONSOLE CARTRIDGE



BY PROF.PLANLOZ

2026-05-20

OPEN THIS PDF WITH PY-16 TO PLAY

PY16OS CART - MANUAL

DESCRIPTION

py16os — A window desktop environment for py-16.

A mini operating system cart featuring a window manager,
taskbar, drag-&-drop, 7 built-in apps, and a plugin
system for custom extensions.

Built-in Apps:
- NOTES: Text editor with on-screen keyboard.
- CMD: DOS-like command line interface.
- PAINT: 32x32 pixel art editor.
- CALC: Pocket calculator.
- THEME: Customize colors and language.
- FILES: File browser with drag & drop support.
- PLAYER: Audio player for MP3/OGG/WAV files.

Drag & Drop Features:
- Drop a .txt file into NOTES to load the text.
- Drop an .mp3/.ogg/.wav file into PLAYER to play it.
- Drop a .p16img file into PAINT to edit the image.
- Drop a .p16img file onto a Desktop Icon to set a custom icon.

Context Menu (Right Click):
- OPEN / MOVE / DELETE to manage files and folders.
- NEW TXT to create a new text document.
- ADD to place installed plugins as icons on the desktop.

Key Terminal (CMD) Commands:
- HELP, LS, CD, CAT, RUN (to start .p16/.pdf carts).
- EDIT (opens file in NOTES), PLAY (plays audio in PLAYER).

Plugin System (apps/ folder):
- Every .py file inside apps/ acts as an independent app.
- Requires an APP dictionary, and init(), update(), and draw() functions.

File Formats & Persistence:
- .p16img: 32x32 hex pixel format used by PAINT.
- theme.json: Persistently saves colors, cursor, language, and desktop icons.

CONTROLS

```
Mouse / D-Pad  : Move cursor
Left Click / Z : Select / Drag / Interact
Right Click / X: Open Context Menu
Start / Enter  : Open Start Menu
```

CREDITS

PY16OS CART - ASSETS

SPRITE SHEET (256x256)

MAP (128x128)

SFX SLOTS

MUSIC TRACKS

PY16OS CART - CODE LISTING (PAGE 1)

```
# @manual
# @description
# py16os — A window desktop environment for py-16.
#
# A mini operating system cart featuring a window manager,
# taskbar, drag-&-drop, 7 built-in apps, and a plugin
# system for custom extensions.
#
# Built-in Apps:
# - NOTES: Text editor with on-screen keyboard.
# - CMD: DOS-like command line interface.
# - PAINT: 32x32 pixel art editor.
# - CALC: Pocket calculator.
# - THEME: Customize colors and language.
# - FILES: File browser with drag & drop support.
# - PLAYER: Audio player for MP3/OGG/WAV files.
#
# Drag & Drop Features:
# - Drop a .txt file into NOTES to load the text.
# - Drop an .mp3/.ogg/.wav file into PLAYER to play it.
# - Drop a .p16img file into PAINT to edit the image.
# - Drop a .p16img file onto a Desktop Icon to set a custom icon.
#
# Context Menu (Right Click):
# - OPEN / MOVE / DELETE to manage files and folders.
# - NEW TXT to create a new text document.
# - ADD to place installed plugins as icons on the desktop.
#
# Key Terminal (CMD) Commands:
# - HELP, LS, CD, CAT, RUN (to start .p16/.pdf carts).
# - EDIT (opens file in NOTES), PLAY (plays audio in PLAYER).
#
# Plugin System (apps/ folder):
# - Every .py file inside apps/ acts as an independent app.
# - Requires an APP dictionary, and init(), update(), and draw() functions.
#
# File Formats & Persistence:
# - .p16img: 32x32 hex pixel format used by PAINT.
# - theme.json: Persistently saves colors, cursor, language, and desktop icons.
#
# @controls
# Mouse / D-Pad  : Move cursor
# Left Click / Z : Select / Drag / Interact
# Right Click / X: Open Context Menu
# Start / Enter  : Open Start Menu
#
# @credits
# Vibe-coder: Prof.Plonloz
# Code: Gemini 3.1 pro & Claud opus 4.7
# @end
import py16
import time
import json
import os
import ast
import sys
import operator
import importlib.util
import pygame

# --- KONFIGURATION & GLOBALS ---
COLOR_WINDOW_BG = 7
COLOR_TITLE_BAR = 1
COLOR_BORDER_LT = 15
```

```
COLOR_BORDER_DK = 5

# --- LAYOUT-KONSTANTEN (zentral, statt verstreuter Magic Numbers) ---
KB_BLOCK_H = 62        # Hoehe des On-Screen-Keyboard-Blocks ab Fensterunterkante
KB_KEY_H = 10          # Hoehe einer Tastenkappe
KB_ROW_PITCH = 12      # Vertikaler Abstand zwischen Tastenreihen
KB_NARROW_W = 10       # Breite einer schmalen Taste (Reihen 0-3)
KB_WIDE_W = 30         # Breite einer breiten Taste (SPACE/<-/ENT)
TASKBAR_H = 12         # Hoehe der Taskleiste

desktop_color = 3
sys_text_color = 1
cursor_color = 7
crt_effect = False     # Ersetzt input_mode für coole Röhren-Effekte!
cursor_x = 128
cursor_y = 112
prev_mx, prev_my = -1, -1  # Für nahtlosen Maus/Gamepad-Wechsel

CONFIG_FILE = "theme.json"

# --- LOKALISIERUNG ---
LANG_DIR = "lang"
current_language = "en"
_translations = {}              # key -> uebersetzter String (leer = englisch)
_available_languages = []       # [{"code": "de", "name": "Deutsch"}]

_EXAMPLE_LANG_DE = {
    "_name": "Deutsch",
    # Kontextmenue
    "OPEN": "OEFFNEN",
    "MOVE": "VERSCHIEBEN",
    "DELETE": "LOESCHEN",
    "CANCEL": "ABBRECHEN",
    "NEW TXT": "NEUE TXT",
    "ADD": "HINZU",
    # Dialoge
    "YES": "JA",
    "NO": "NEIN",
    "RUN": "STARTEN",
    # THEME-Labels
    "DESKTOP:": "HINTERGRUND:",
    "TEXT:": "TEXT:",
    "CURSOR:": "CURSOR:",
    "LANGUAGE:": "SPRACHE:",
}

def tr(key):
    """Liefert die Uebersetzung oder den Original-Key (Fallback)."""
    return _translations.get(key, key)

def discover_languages():
    """Scannt lang/ und befuellt _available_languages. Legt beim Erststart eine de.json an
    global _available_languages
    _available_languages = [{"code": "en", "name": "English"}]
    try:
        if not os.path.isdir(LANG_DIR):
            os.makedirs(LANG_DIR, exist_ok=True)
            with open(os.path.join(LANG_DIR, "de.json"), "w") as f:
                json.dump(_EXAMPLE_LANG_DE, f, indent=2, ensure_ascii=False)
        for fn in sorted(os.listdir(LANG_DIR)):
            if not fn.lower().endswith(".json"):
                continue
            code = os.path.splitext(fn)[0].lower()
            if code == "en":
```

```
                continue
            try:
                with open(os.path.join(LANG_DIR, fn)) as f:
                    data = json.load(f)
                name = data.get("_name", code.upper())
                _available_languages.append({"code": code, "name": str(name)})
            except Exception:
                pass
    except Exception:
        pass

def load_language(code):
    """Laedt eine Sprachdatei in _translations. 'en' = leere Map (Fallback auf Keys)."""
    global _translations, current_language
    code = (code or "en").lower()
    current_language = code
    _translations = {}
    if code == "en":
        return True
    path = os.path.join(LANG_DIR, code + ".json")
    if not os.path.isfile(path):
        current_language = "en"
        return False
    try:
        with open(path) as f:
            data = json.load(f)
        _translations = {k: str(v) for k, v in data.items() if k != "_name"}
        return True
    except Exception:
        current_language = "en"
        return False

def context_label(opt):
    """Uebersetzt eine Kontextmenue-Option fuer die Anzeige.
    Interner Vergleich findet weiter auf den englischen Tokens statt."""
    if opt.startswith("ADD "):
        return tr("ADD") + " " + opt[4:]
    return tr(opt)

def load_theme():
    global desktop_color, sys_text_color, cursor_color, crt_effect, desktop_icons, known_p
    if os.path.exists(CONFIG_FILE):
        try:
            with open(CONFIG_FILE, "r") as f:
                data = json.load(f)
                desktop_color = data.get("desktop_color", desktop_color)
                sys_text_color = data.get("sys_text_color", sys_text_color)
                cursor_color = data.get("cursor_color", cursor_color)
                crt_effect = data.get("crt_effect", crt_effect)
                if "desktop_icons" in data:
                    desktop_icons = data["desktop_icons"]
                if "known_plugins" in data:
                    known_plugins = list(data["known_plugins"])
                if "language" in data:
                    current_language = data["language"]
        except Exception: pass

def save_theme():
    data = {"desktop_color": desktop_color, "sys_text_color": sys_text_color,
            "cursor_color": cursor_color, "crt_effect": crt_effect,
            "desktop_icons": desktop_icons, "known_plugins": known_plugins,
            "language": current_language}
    try:
        with open(CONFIG_FILE, "w") as f: json.dump(data, f)
```

```
    except Exception: pass

# --- FENSTER STATE ---
windows = [
    {"id": "notepad", "title": "NOTES.PY16", "x": 20, "y": 20, "w": 140, "h": 130, "visibl
    {"id": "files", "title": "FILES.PY16", "x": 35, "y": 30, "w": 160, "h": 130, "visible"
     "scroll": 0, "selected": "", "items": [], "current_dir": ".", "is_sel_dir": False, "m
    {"id": "colors", "title": "THEME.PY16", "x": 60, "y": 60, "w": 116, "h": 156, "visible
    {"id": "calc", "title": "CALC.PY16", "x": 150, "y": 40, "w": 88, "h": 110, "visible":
     "disp": "0", "val": 0, "op": None, "new_num": True, "pressed_btn": -1},
    {"id": "paint", "title": "PAINT.PY16", "x": 10, "y": 80, "w": 100, "h": 88, "visible":
    {"id": "terminal", "title": "CMD.PY16", "x": 80, "y": 20, "w": 150, "h": 120, "visible
    {"id": "music", "title": "PLAYER.PY16", "x": 100, "y": 80, "w": 130, "h": 120, "visibl
]

# Globals für Drag & Drop und Resize
dragged_window, drag_off_x, drag_off_y = None, 0, 0
resized_window, resize_start_w, resize_start_h, resize_start_mx, resize_start_my = None, 0
start_menu_open, date_popup_open = False, False
dragged_file = None
selected_desktop_icon = None

# Neue Globals für das Kontextmenü
context_menu_open = False
context_menu_x, context_menu_y = 0, 0
context_menu_options = [] # Speichert die klickbaren Optionen dynamisch

# Modaler Bestätigungsdialog: None oder {"text": str, "on_yes": callable}
confirm_dialog = None

def ask_confirm(text, on_yes):
    """Oeffnet einen modalen JA/NEIN-Dialog. on_yes() wird bei JA aufgerufen."""
    global confirm_dialog
    confirm_dialog = {"text": text, "on_yes": on_yes}

# --- Sicherer Arithmetik-Parser (ersetzt das gefaehrliche eval()) ---
_SAFE_OPS = {
    ast.Add: operator.add, ast.Sub: operator.sub,
    ast.Mult: operator.mul, ast.Div: operator.truediv,
    ast.Mod: operator.mod, ast.Pow: operator.pow,
    ast.USub: operator.neg, ast.UAdd: operator.pos,
}

def _safe_eval_node(node):
    if isinstance(node, ast.Expression):
        return _safe_eval_node(node.body)
    if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in _SAFE_OPS:
        return _SAFE_OPS[type(node.op)](_safe_eval_node(node.left),
                                        _safe_eval_node(node.right))
    if isinstance(node, ast.UnaryOp) and type(node.op) in _SAFE_OPS:
        return _SAFE_OPS[type(node.op)](_safe_eval_node(node.operand))
    raise ValueError("unsupported expression")

def safe_eval(expr):
    """Wertet nur reine Arithmetik aus: + - * / % ** ( ) und Zahlen."""
    return _safe_eval_node(ast.parse(expr, mode="eval"))

# --- Gemeinsame On-Screen-Tastatur (war doppelt in Notepad + Terminal) ---
def draw_keyboard(win, wx, wy, ww, wh):
    kb_y = wy + wh - KB_BLOCK_H
    py16.line(wx+4, kb_y - 4, wx+ww-5, kb_y - 4, 5)
    for r_idx, row in enumerate(KB_ROWS):
```

```
        row_w = len(row) * KB_ROW_PITCH if r_idx < 4 else 3 * 32
        start_x = wx + (ww - row_w) // 2
        for k_idx, key in enumerate(row):
            kw = KB_NARROW_W if r_idx < 4 else KB_WIDE_W
            kx = start_x + k_idx * (kw + 2)
            ky = kb_y + r_idx * KB_ROW_PITCH
            is_pressed = win.get("pressed_key") == key
            py16.rectfill(kx, ky, kw, KB_KEY_H, 5 if is_pressed else 6)
            if not is_pressed:
                py16.line(kx, ky, kx+kw-1, ky, 15); py16.line(kx, ky, kx, ky+9, 15)
                py16.line(kx+1, ky+9, kx+kw-1, ky+9, 5); py16.line(kx+kw-1, ky+1, kx+kw-1,
            py16.rect(kx, ky, kw, KB_KEY_H, 0)
            py16.text(key, kx + (kw - len(key) * 4)//2 + 1, ky + 3,
                      7 if is_pressed else sys_text_color)

def keyboard_hit(win, lx, ly, m_held):
    """Liefert die getroffene Taste (String) oder None. Setzt pressed_key bei Halten."""
    kb_y = win["h"] - KB_BLOCK_H
    if ly < kb_y - 4:
        return None
    r_idx = (ly - kb_y) // KB_ROW_PITCH
    if not (0 <= r_idx < len(KB_ROWS)):
        return None
    row = KB_ROWS[r_idx]
    row_w = len(row) * KB_ROW_PITCH if r_idx < 4 else 3 * 32
    start_x = (win["w"] - row_w) // 2
    k_idx = (lx - start_x) // KB_ROW_PITCH if r_idx < 4 else (lx - start_x) // 32
    if 0 <= k_idx < len(row) and start_x <= lx <= start_x + row_w:
        key = row[k_idx]
        if m_held:
            win["pressed_key"] = key
        return key
    return None

def is_dir_item(win, name):
    """Verzeichnis-Check ohne os.stat: nutzt den in load_files gecachten dir_set."""
    return name == "[ .. ]" or name in win.get("dir_set", set())

DEFAULT_APPS = [
    {"id": "files", "name": "FILES"},
    {"id": "notepad", "name": "NOTES"},
    {"id": "paint", "name": "PAINT"},
    {"id": "music", "name": "PLAYER"},
    {"id": "terminal", "name": "CMD"},
    {"id": "calc", "name": "CALC"},
    {"id": "colors", "name": "THEME"}
]
desktop_icons = list(DEFAULT_APPS) # Standard-Icons beim Start kopieren
known_plugins = []                 # IDs aller Plugins, die das System je gesehen hat

# Keyboard Layout
KB_ROWS = [
    list("1234567890"),
    list("QWERTZUIOP"),
    list("ASDFGHJKL-"),
    list("YXCVBNM.,!"),
    ["SPACE", "<-", "ENT"]
]

# ============================================================================
# === APPS (jede App hat eine *_draw und eine *_update Funktion) ============
# ============================================================================

# -- 1. NOTEPAD ------------------------------------------------------------
```

```
# Einfacher Texteditor mit On-Screen-Tastatur.
#
# LAYOUT (von oben nach unten):
#   * Werkzeugleiste: [NEW] [SAVE]   (Toolbar-Buttons unter der Titelleiste)
#   * Textbereich mit vertikalem Scrollbalken am rechten Rand
#   * Trennlinie
#   * On-Screen-Tastatur (siehe draw_keyboard) im unteren Drittel
#
# BEDIENUNG:
#   * Tasten antippen -> Zeichen wird an die letzte Zeile angehängt.
#   * ENT/Enter      -> neue Zeile.
#   * <-/Backspace   -> letztes Zeichen löschen.
#   * NEW            -> Inhalt leeren, neuer Datei-Slot.
#   * SAVE           -> aktuell geladene Datei überschreiben (falls vorhanden).
#   * Drag-&-Drop einer Textdatei aus FILES auf das Fenster lädt sie hier.
#
# WIN-STATE:
#   lines: list[str], pressed_key: str|None, filename: str|None, scroll: int
def notepad_draw(win, wx, wy, ww, wh, is_active):
    py16.rectfill(wx+6, wy+14, 18, 9, 6); py16.rect(wx+6, wy+14, 18, 9, 0); py16.text("NEW
    py16.rectfill(wx+26, wy+14, 22, 9, 6); py16.rect(wx+26, wy+14, 22, 9, 0); py16.text("S
    py16.line(wx+4, wy+25, wx+ww-5, wy+25, 5)

    text_h, sb_x = wh - 88, wx + ww - 14
    visible_lines, scroll = max(1, text_h // 10), win.get("scroll", 0)

    py16.rectfill(sb_x, wy+36, 10, text_h - 20, 6)
    py16.rectfill(sb_x, wy+26, 10, 10, 6); py16.rect(sb_x, wy+26, 10, 10, 0); py16.text("^
    py16.rectfill(sb_x, wy+26+text_h-10, 10, 10, 6); py16.rect(sb_x, wy+26+text_h-10, 10,

    py16.clip(wx+4, wy+26, ww-20, text_h)
    lines = win.get("lines", [""])
    for i in range(visible_lines):
        idx = scroll + i
        if idx < len(lines): py16.text(lines[idx], wx+6, wy+28 + i*10, sys_text_color)

    if (py16.t() // 30) % 2 == 0 and is_active:
        active_line_idx = len(lines) - 1
        if scroll <= active_line_idx < scroll + visible_lines:
            cx, cy = wx + 6 + len(lines[-1]) * 4, wy + 28 + (active_line_idx - scroll) * 1
            py16.rectfill(cx, cy, 4, 6, sys_text_color)
    py16.clip()

    draw_keyboard(win, wx, wy, ww, wh)

def notepad_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    win["pressed_key"] = None
    if not (m_pressed or m_sec_pressed or m_held): return

    text_h = win["h"] - 88
    visible_lines = max(1, text_h // 10)

    if (m_pressed or m_sec_pressed) and 14 <= ly <= 23:
        if 6 <= lx <= 24:
            win["lines"], win["filename"], win["title"], win["scroll"] = [""], None, "NOTE
            py16.tone(880, 10, py16.WAVE_SQUARE); return
        elif 26 <= lx <= 48:
            if not win.get("filename"):
                i = 1
                while os.path.exists(f"note{i}.txt"): i += 1
                win["filename"], win["title"] = f"note{i}.txt", f"NOTES [{f'note{i}.txt'}]
            try:
                with open(win["filename"], "w") as f: f.write("\n".join(win.get("lines", [
                py16.tone(880, 20, py16.WAVE_SQUARE)
```

```
            except Exception: py16.tone(200, 20, py16.WAVE_SAW)
            return

    sb_x = win["w"] - 14
    if (m_pressed or m_sec_pressed) and sb_x <= lx <= sb_x + 10:
        if 26 <= ly <= 36: win["scroll"] = max(0, win.get("scroll", 0) - 1); py16.tone(440
        elif 26 + text_h - 10 <= ly <= 26 + text_h:
            win["scroll"] = min(max(0, len(win.get("lines", [""])) - visible_lines), win.g

    kb_y = win["h"] - KB_BLOCK_H
    if ly < kb_y - 4: return
    key = keyboard_hit(win, lx, ly, m_held)
    if key is not None:
            if m_pressed or m_sec_pressed:
                py16.tone(880, 10, py16.WAVE_SQUARE)
                lines = win.get("lines", [""])
                max_chars = (win["w"] - 22) // 4

                if key == "<-":
                    if len(lines[-1]) > 0: lines[-1] = lines[-1][:-1]
                    elif len(lines) > 1:
                        lines.pop()
                        if win.get("scroll", 0) > max(0, len(lines) - visible_lines): win[
                elif key == "ENT":
                    lines.append("")
                    if len(lines) > visible_lines + win.get("scroll", 0): win["scroll"] =
                elif key == "SPACE":
                    if len(lines[-1]) >= max_chars:
                        lines.append("")
                        if len(lines) > visible_lines + win.get("scroll", 0): win["scroll"
                    else: lines[-1] += " "
                else:
                    if len(lines[-1]) >= max_chars:
                        last_space = lines[-1].rfind(" ")
                        if last_space != -1:
                            word = lines[-1][last_space+1:] + key; lines[-1] = lines[-1][:
                        else: lines.append(key)
                        if len(lines) > visible_lines + win.get("scroll", 0): win["scroll"
                    else: lines[-1] += key
                win["lines"] = lines

# -- 2. TERMINAL (CLI) ------------------------------------------------------
# DOS-artige Kommandozeile. Eigene cwd, unabhängig vom FILES-Fenster.
#
# AVAILABLE COMMANDS (siehe HELP):
#   * Info:        HELP, VER, WHOAMI, TIME, DATE, ECHO, CLEAR/CLS, THEME, CALC
#   * Navigation:  PWD/CWD, LS/DIR, CD <pfad>
#   * Dateien:     CAT/TYPE <datei>, TOUCH/NEW <datei>, MKDIR <ordner>
#                  EDIT/OPEN <datei> (öffnet in NOTES)
#                  PLAY <datei>      (spielt im PLAYER ab, MP3/OGG/WAV)
#                  RUN <cart.p16>    (startet eine py-16-Cart)
#   * Löschen:     RM/DEL <datei>, RMDIR <ordner>  — mit Bestätigung
#   * Plugins:     APPS (listet geladene Plugins), RELOAD (apps/ neu scannen)
#
# SICHERHEIT:
#   * CALC nutzt safe_eval() (keine eval()-Code-Execution).
#   * RM/RMDIR/RUN gehen durch den modalen Bestätigungsdialog.
#
# WIN-STATE:
#   lines: list[str] (Verlauf), input_str: str (aktuelle Eingabezeile),
#   scroll: int, cwd: str (Arbeitsverzeichnis), pressed_key: str|None
def open_in_notepad(path):
    """Laedt eine Textdatei ins NOTES-Fenster und holt es nach vorn."""
    notepad = next((w for w in windows if w["id"] == "notepad"), None)
```

```
    if not notepad: return False
    try:
        with open(path, "r") as f: content = f.read().replace('\r', '')
        notepad["lines"] = content.split('\n') if content else [""]
        notepad["filename"], notepad["title"] = path, f"NOTES [{os.path.basename(path)}]"
        notepad["scroll"], notepad["visible"], notepad["minimized"] = 0, True, False
        windows.remove(notepad); windows.append(notepad); py16.tone(880, 20, py16.WAVE_SQU
        return True
    except Exception:
        py16.tone(200, 20, py16.WAVE_SAW); return False

def play_in_music(path):
    """Spielt eine Audiodatei im PLAYER-Fenster ab und holt es nach vorn."""
    music = next((w for w in windows if w["id"] == "music"), None)
    if not music: return False
    if "playlist" not in music: music["playlist"] = []
    music["playlist"].append({"name": os.path.basename(path), "path": path})
    music["mode"], music["file_name"], music["file_path"], music["playing"] = "external",
    py16.music(-1)
    try:
        pygame.mixer.music.load(path); pygame.mixer.music.play()
        music["visible"], music["minimized"] = True, False
        windows.remove(music); windows.append(music); py16.tone(880, 20, py16.WAVE_SQUARE)
        return True
    except Exception:
        py16.tone(200, 20, py16.WAVE_SAW)
        music["playing"], music["file_name"] = False, "PYGAME ERR"
        return False

def _term_path(win, arg):
    """Loest ein Argument relativ zum Terminal-cwd auf."""
    cwd = win.setdefault("cwd", os.path.abspath("."))
    return arg if os.path.isabs(arg) else os.path.join(cwd, arg)

CART_EXTS = ('.p16', '.pdf')

def is_cart_file(path):
    """True, wenn die Datei eine startbare py-16-Cart ist (.p16 oder .pdf)."""
    return path.lower().endswith(CART_EXTS)

def launch_cart(path):
    """Startet eine .p16/.pdf-Cart. push_cart merkt sich das aktuelle OS,
    sodass die Cart zum Desktop zurueckkehren kann; sonst run_cart als Fallback."""
    try:
        pygame.mixer.music.stop()
    except Exception:
        pass
    py16.music(-1)
    try:
        py16.push_cart(path)            # OS merken, Cart starten
        return True
    except Exception:
        try:
            py16.run_cart(path)         # Fallback: ersetzen
            return True
        except Exception:
            py16.tone(200, 25, py16.WAVE_SAW)
            return False

def ask_launch(path):
    """Cart-Start ueber den modalen Bestaetigungsdialog (ein Sicherheitsmodell)."""
    ask_confirm(tr("RUN") + " " + os.path.basename(path)[:16] + "?", lambda p=path: launch

_TERM_HELP = [
```

```
    "AVAILABLE CMDS:",
    " HELP CLEAR CLS VER",
    " ECHO TIME DATE WHOAMI",
    " PWD LS DIR CD",
    " CAT TYPE TOUCH NEW",
    " MKDIR RM DEL RMDIR",
    " EDIT OPEN PLAY RUN",
    " APPS RELOAD",
    " CALC THEME",
]

def execute_terminal_cmd(win, cmd_str):
    cmd = cmd_str.strip()
    if not cmd: return
    parts = cmd.split(" ")
    base, args = parts[0].upper(), parts[1:]   # nur das Kommando uppercasen
    arg_str = " ".join(args)
    out = win["lines"]
    win.setdefault("cwd", os.path.abspath("."))

    if base == "HELP":
        out.extend(_TERM_HELP)
    elif base in ("CLEAR", "CLS"):
        win["lines"] = ["PY16 DOS V1.1", "READY."]
    elif base == "VER":
        out.append("PY16 DOS V1.1")
    elif base == "WHOAMI":
        out.append("PY16USER")
    elif base == "ECHO":
        out.append(arg_str)
    elif base == "TIME":
        t = time.localtime()
        out.append(f"TIME: {t.tm_hour:02d}:{t.tm_min:02d}:{t.tm_sec:02d}")
    elif base == "DATE":
        t = time.localtime()
        days = ["MON", "TUE", "WED", "THU", "FRI", "SAT", "SUN"]
        out.append(f"DATE: {days[t.tm_wday]} {t.tm_mday:02d}.{t.tm_mon:02d}.{t.tm_year}")
    elif base in ("PWD", "CWD"):
        p = win["cwd"]
        out.append(p if len(p) <= 36 else "..." + p[-33:])
    elif base in ("LS", "DIR"):
        try:
            raw = sorted(os.listdir(win["cwd"]))
            dirs = [n for n in raw if os.path.isdir(os.path.join(win["cwd"], n))]
            files = [n for n in raw if n not in dirs]
            shown = 0
            for n in dirs + files:
                if shown >= 60:
                    out.append(f"...(+{len(raw) - shown} MORE)"); break
                tag = "<DIR> " if n in dirs else "      "
                out.append(tag + n[:24]); shown += 1
            if not raw: out.append("(EMPTY)")
        except Exception:
            out.append("CANNOT LIST DIR")
    elif base == "CD":
        target = os.path.abspath(_term_path(win, arg_str) if arg_str else os.path.abspath(
        if os.path.isdir(target):
            win["cwd"] = target
        else:
            out.append("NO SUCH DIR")
    elif base in ("CAT", "TYPE"):
        if not arg_str:
            out.append("USAGE: CAT <FILE>")
        else:
```

```
            p = _term_path(win, arg_str)
            try:
                with open(p, "r") as f:
                    flines = f.read().replace('\r', '').split('\n')
                for ln in flines[:100]:
                    out.append(ln)
                if len(flines) > 100:
                    out.append(f"...(+{len(flines) - 100} LINES)")
            except Exception:
                out.append("CANNOT READ FILE")
    elif base in ("TOUCH", "NEW"):
        if not arg_str:
            out.append("USAGE: TOUCH <FILE>")
        else:
            p = _term_path(win, arg_str)
            try:
                if os.path.exists(p): out.append("ALREADY EXISTS")
                else:
                    open(p, "w").close(); out.append("CREATED " + os.path.basename(p))
            except Exception:
                out.append("CANNOT CREATE FILE")
    elif base == "MKDIR":
        if not arg_str:
            out.append("USAGE: MKDIR <DIR>")
        else:
            p = _term_path(win, arg_str)
            try:
                os.mkdir(p); out.append("CREATED " + os.path.basename(p))
            except Exception:
                out.append("CANNOT CREATE DIR")
    elif base in ("RM", "DEL", "RMDIR"):
        if not arg_str:
            out.append(f"USAGE: {base} <NAME>")
        else:
            p = _term_path(win, arg_str)
            want_dir = (base == "RMDIR")
            if not os.path.exists(p):
                out.append("NOT FOUND")
            elif want_dir and not os.path.isdir(p):
                out.append("NOT A DIR")
            elif not want_dir and os.path.isdir(p):
                out.append("IS A DIR (USE RMDIR)")
            else:
                def _do_rm(pp=p, w=win, is_dir=want_dir):
                    try:
                        os.rmdir(pp) if is_dir else os.remove(pp)
                        w["lines"].append("DELETED " + os.path.basename(pp))
                        py16.tone(150, 25, py16.WAVE_SAW)
                    except Exception:
                        w["lines"].append("DELETE FAILED")
                        py16.tone(100, 30, py16.WAVE_NOISE)
                ask_confirm(tr("DELETE") + " " + os.path.basename(p)[:16] + "?", _do_rm)
    elif base in ("EDIT", "OPEN"):
        if not arg_str:
            out.append("USAGE: EDIT <FILE>")
        else:
            p = _term_path(win, arg_str)
            if not os.path.isfile(p): out.append("NO SUCH FILE")
            elif not open_in_notepad(p): out.append("CANNOT OPEN")
    elif base == "PLAY":
        if not arg_str:
            out.append("USAGE: PLAY <FILE>")
        else:
            p = _term_path(win, arg_str)
```

```
            if not os.path.isfile(p):
                out.append("NO SUCH FILE")
            elif not p.lower().endswith(('.mp3', '.ogg', '.wav')):
                out.append("NOT AUDIO (MP3/OGG/WAV)")
            else:
                play_in_music(p); out.append("PLAYING " + os.path.basename(p)[:18])
    elif base == "RUN":
        if not arg_str:
            out.append("USAGE: RUN <CART.P16>")
        else:
            p = _term_path(win, arg_str)
            if not os.path.isfile(p):
                out.append("NO SUCH FILE")
            elif not is_cart_file(p):
                out.append("NOT A CART (.P16/.PDF)")
            else:
                out.append("LAUNCHING " + os.path.basename(p)[:16])
                ask_launch(p)
    elif base == "APPS":
        if PLUGIN_APPS:
            out.append("PLUGINS:")
            for a in PLUGIN_APPS:
                out.append(" " + a["name"] + " (" + a["id"] + ")")
        else:
            out.append("NO PLUGINS IN apps/")
    elif base == "RELOAD":
        out.append("SCANNING apps/ ...")
        out.extend(load_plugins())
    elif base == "THEME":
        out.append("TIP: USE THEME APP")
    elif base == "CALC":
        try: out.append(f"= {safe_eval(arg_str)}")
        except Exception: out.append("SYNTAX ERROR")
    else:
        out.append(f"BAD CMD: {base}")

def terminal_draw(win, wx, wy, ww, wh, is_active):
    text_h, sb_x = wh - 76, wx + ww - 14
    term_c = 11
    py16.rectfill(wx+4, wy+14, ww-18, text_h, 0); py16.rect(wx+3, wy+13, ww-16, text_h+2,

    visible_lines, scroll = max(1, text_h // 10), win.get("scroll", 0)
    py16.rectfill(sb_x, wy+24, 10, text_h - 20, 6)
    py16.rectfill(sb_x, wy+14, 10, 10, 6); py16.rect(sb_x, wy+14, 10, 10, 0); py16.text("^
    py16.rectfill(sb_x, wy+14+text_h-10, 10, 10, 6); py16.rect(sb_x, wy+14+text_h-10, 10,

    py16.clip(wx+4, wy+14, ww-18, text_h)
    lines_to_draw = win["lines"] + ["> " + win["input_str"]]
    for i in range(visible_lines):
        idx = scroll + i
        if idx < len(lines_to_draw): py16.text(lines_to_draw[idx], wx+6, wy+16 + i*10, ter

    if (py16.t() // 20) % 2 == 0 and is_active:
        active_line_idx = len(lines_to_draw) - 1
        if scroll <= active_line_idx < scroll + visible_lines:
            cx, cy = wx + 6 + len(lines_to_draw[-1]) * 4, wy + 16 + (active_line_idx - scr
            py16.rectfill(cx, cy, 4, 6, term_c)
    py16.clip()

    draw_keyboard(win, wx, wy, ww, wh)

def terminal_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    win["pressed_key"] = None
    if not (m_pressed or m_sec_pressed or m_held): return
```

```
705
706     text_h, lines_total, sb_x = win["h"] - 76, len(win["lines"]) + 1, win["w"] - 14
707     visible_lines = max(1, text_h // 10)
708
709     if (m_pressed or m_sec_pressed) and sb_x <= lx <= sb_x + 10:
710         if 14 <= ly <= 24: win["scroll"] = max(0, win.get("scroll", 0) - 1); py16.tone(440
711         elif 14 + text_h - 10 <= ly <= 14 + text_h:
712             win["scroll"] = min(max(0, lines_total - visible_lines), win.get("scroll", 0)
713
714     kb_y = win["h"] - KB_BLOCK_H
715     if ly < kb_y - 4: return
716     key = keyboard_hit(win, lx, ly, m_held)
717     if key is not None:
718             if m_pressed or m_sec_pressed:
719                 py16.tone(440, 5, py16.WAVE_SQUARE)
720                 max_chars = (win["w"] - 30) // 4
721                 if key == "<-": win["input_str"] = win["input_str"][:-1]
722                 elif key == "ENT":
723                     win["lines"].append("> " + win["input_str"]); execute_terminal_cmd(win
724                     if len(win["lines"]) + 1 > visible_lines: win["scroll"] = len(win["lin
725                 elif key == "SPACE":
726                     if len(win["input_str"]) < max_chars: win["input_str"] += " "
727                 else:
728                     if len(win["input_str"]) < max_chars: win["input_str"] += key
729                 if win["scroll"] < lines_total - visible_lines: win["scroll"] = max(0, lin
730
731 # -- 3. PAINT ------------------------------------------------------------------
732 # Pixel-Editor: 32x32 Canvas, 16-Farben-Palette, Speichert/Lädt .p16img.
733 #
734 # LAYOUT:
735 #   * Canvas links: 64x64 Pixel (32 logische Pixel zu je 2x2 Bildpunkten)
736 #   * Palette rechts: 16 Farben in 2 Spalten x 8 Reihen
737 #   * Buttons unten: [C] Clear / [S] Save / [L] Load-Hinweis
738 #   * Statuszeile unter den Buttons (zeigt SAVED ... etc.)
739 #
740 # BEDIENUNG:
741 #   * Pixel anklicken (auch ziehen) -> mit aktueller Farbe füllen
742 #   * Palettenfarbe anklicken -> Farbe wechseln
743 #   * C  -> Canvas zurücksetzen
744 #   * S  -> Bild als img_NNN.p16img im FILES-Verzeichnis speichern
745 #   * L  -> Zeigt nur den Hinweis "DROP .P16IMG HERE" — Laden geht via D&D
746 #   * Drag-&-Drop einer .p16img aus FILES auf das Fenster lädt das Bild
747 #   * Drag-&-Drop einer .p16img aus FILES auf ein Desktop-Icon weist sie
748 #     als App-Icon zu (persistent in theme.json)
749 #
750 # .p16img-FORMAT:
751 #   * Klartext: 32 Zeilen mit je 32 Hex-Zeichen (0-F = Palettenindex)
752 #   * Kommentarzeilen mit '#' am Anfang werden ignoriert
753 #   * Farbe 7 gilt beim Icon-Rendern als transparent
754
755 # .p16img-Format: 32 Zeilen, jede Zeile 32 Hex-Zeichen (0-F) = ein Palettenindex pro Pixel
756 # Kommentare ('#') und Leerzeilen werden beim Laden ignoriert.
757
758 _image_cache = {}  # path -> Liste mit 1024 Farbindizes (oder None bei Fehler)
759
760 def save_p16img(path, canvas):
761     """Speichert eine 32x32-Canvas als .p16img-Datei. Liefert True bei Erfolg."""
762     try:
763         with open(path, "w") as f:
764             f.write("# P16IMG 32x32\n")
765             for row in range(32):
766                 line = "".join(format(canvas[row * 32 + col] & 0x0F, "X") for col in range
767                 f.write(line + "\n")
768         _image_cache[os.path.abspath(path)] = list(canvas)
```

```
        return True
    except Exception:
        return False

def load_p16img(path):
    """Laedt .p16img und liefert 1024-elementige Liste; bei Fehler None. Mit Cache."""
    key = os.path.abspath(path)
    if key in _image_cache:
        return _image_cache[key]
    try:
        with open(path, "r") as f:
            raw = f.read()
        pixels = []
        for line in raw.splitlines():
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            for ch in line[:32]:
                if ch in "0123456789ABCDEFabcdef":
                    pixels.append(int(ch, 16))
                else:
                    break
            if len(pixels) >= 1024:
                break
        if len(pixels) < 1024:
            pixels.extend([7] * (1024 - len(pixels)))
        else:
            pixels = pixels[:1024]
        _image_cache[key] = pixels
        return pixels
    except Exception:
        _image_cache[key] = None
        return None

def draw_icon_image(path, ix, iy):
    """Zeichnet ein .p16img als 16x16-Icon (32x32 -> jeder 2. Pixel). Liefert True bei Erf
    px = load_p16img(path)
    if px is None:
        return False
    for ry in range(16):
        for rx in range(16):
            c = px[(ry * 2) * 32 + (rx * 2)]
            if c != 7:                          # 7 = transparent (Canvas-Hintergrund)
                py16.pset(ix + 2 + rx, iy + ry, c)
    return True

def next_image_filename(directory):
    """Findet den naechsten freien IMG_NNN.P16IMG-Pfad."""
    for n in range(1, 1000):
        cand = os.path.join(directory, f"img_{n:03d}.p16img")
        if not os.path.exists(cand):
            return cand
    return os.path.join(directory, "img_overflow.p16img")

# -- 3.b PAINT: Render + Update ------------------------------------------------
def paint_draw(win, wx, wy, ww, wh, is_active):
    cx, cy = wx + 6, wy + 16
    py16.rect(cx - 1, cy - 1, 66, 66, 0)
    for i in range(1024):
        if win["canvas"][i] != 7: py16.rectfill(cx + (i % 32) * 2, cy + (i // 32) * 2, 2,

    pal_x, pal_y = wx + 76, wy + 16
    for i in range(16):
        rx, ry = pal_x + (i % 2) * 10, pal_y + (i // 2) * 8
```

```
        py16.rectfill(rx, ry, 8, 6, i)
        if win["color"] == i: py16.rect(rx-1, ry-1, 10, 8, 0)
    # Drei kleine Buttons in einer Reihe: CLR / SAV / LOD
    for i, label in enumerate(("CLR", "SAV", "LOD")):
        bx = wx + 76 + i * 7
        py16.rectfill(bx, wy + 72, 6, 8, 5); py16.rect(bx, wy + 72, 6, 8, 0)
        py16.text(label[0], bx + 1, wy + 74, 7)
    # Statuszeile unterhalb der Canvas
    if win.get("status"):
        py16.text(str(win["status"])[:24], wx + 6, wy + 82, sys_text_color)

def paint_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    if not (m_pressed or m_sec_pressed or m_held): return
    if 6 <= lx < 70 and 16 <= ly < 80 and (m_held or m_pressed):
        px, py_ = (lx - 6) // 2, (ly - 16) // 2
        if 0 <= px < 32 and 0 <= py_ < 32: win["canvas"][py_ * 32 + px] = win["color"]
    if m_pressed or m_sec_pressed:
        if 76 <= lx < 96 and 16 <= ly < 80:
            pal_idx = ((lx - 76) // 10) + ((ly - 16) // 8) * 2
            if 0 <= pal_idx < 16: win["color"] = pal_idx; py16.tone(880, 10, py16.WAVE_TRI
        # Buttons: CLR (76..81), SAV (83..88), LOD (90..95) auf y=72..79
        if 72 <= ly <= 79:
            if 76 <= lx <= 81:                                  # CLR
                win["canvas"] = [7] * 1024
                win["status"] = "CLEARED"
                py16.tone(440, 20, py16.WAVE_SQUARE)
            elif 83 <= lx <= 88:                                # SAV
                files_w = next((w for w in windows if w["id"] == "files"), None)
                tgt_dir = files_w["current_dir"] if files_w else os.path.abspath(".")
                path = next_image_filename(tgt_dir)
                if save_p16img(path, win["canvas"]):
                    win["status"] = "SAVED " + os.path.basename(path)
                    py16.tone(880, 20, py16.WAVE_SQUARE)
                    if files_w: load_files(files_w)
                else:
                    win["status"] = "SAVE FAILED"
                    py16.tone(200, 25, py16.WAVE_SAW)
            elif 90 <= lx <= 95:                                # LOD: Hinweis statt komplex
                win["status"] = "DROP .P16IMG HERE"
                py16.tone(660, 15, py16.WAVE_TRIANGLE)

# -- 4. CALC -------------------------------------------------------------
# Taschenrechner mit 4x4-Tastenfeld.
#
# LAYOUT:
#   * Display oben (zeigt aktuelle Eingabe oder letztes Ergebnis)
#   * 16 Tasten in 4x4: 7 8 9 / | 4 5 6 * | 1 2 3 - | C 0 = +
#
# BEDIENUNG:
#   * Ziffer/Operator -> hängt an die Eingabe an
#   * =               -> wertet über safe_eval() aus
#   * C               -> löscht die Eingabe
#
# Die Berechnung verwendet denselben sicheren AST-Parser wie CALC im Terminal.
# Division durch null gibt still 0 zurück (kein Crash).
def handle_calc_input(win, idx):
    buttons = ["7","8","9","/", "4","5","6","*", "1","2","3","-", "C","0","=","+"]
    if idx < 0 or idx >= len(buttons): return
    btn = buttons[idx]
    py16.tone(880, 20, py16.WAVE_SQUARE)
    if btn in "0123456789":
        if win["new_num"]: win["disp"], win["new_num"] = btn, False
        elif len(win["disp"]) < 8: win["disp"] = btn if win["disp"] == "0" else win["disp"
    elif btn == "C": win["disp"], win["val"], win["op"], win["new_num"] = "0", 0, None, Tr
```

```
    elif btn in "+-*/": win["val"], win["op"], win["new_num"] = float(win["disp"]), btn, T
    elif btn == "=" and win["op"] is not None:
        v2 = float(win["disp"])
        res = win["val"] + v2 if win["op"] == "+" else win["val"] - v2 if win["op"] == "-"
        res_str = str(res)
        if res_str.endswith(".0"): res_str = res_str[:-2]
        win["disp"], win["val"], win["op"], win["new_num"] = res_str[:8], res, None, True

def calc_draw(win, wx, wy, ww, wh, is_active):
    py16.rectfill(wx+6, wy+16, ww-12, 14, 5); py16.rectfill(wx+7, wy+17, ww-14, 12, 7)
    py16.text(win["disp"], wx + ww - 8 - len(win["disp"]) * 4, wy+21, sys_text_color)
    buttons = ["7","8","9","/", "4","5","6","*", "1","2","3","-", "C","0","=","+"]
    for by in range(4):
        for bx in range(4):
            idx, btn_x, btn_y = by * 4 + bx, wx + 6 + bx*20, wy + 34 + by*18
            if win["pressed_btn"] == idx:
                py16.rectfill(btn_x, btn_y, 16, 14, 5); py16.rect(btn_x, btn_y, 16, 14, 0)
                py16.text(buttons[idx], btn_x + (7 if len(buttons[idx]) == 1 else 5), btn_
            else:
                py16.rectfill(btn_x, btn_y, 16, 14, 6)
                py16.line(btn_x, btn_y, btn_x+15, btn_y, 15); py16.line(btn_x, btn_y, btn_
                py16.line(btn_x, btn_y+13, btn_x+15, btn_y+13, 5); py16.line(btn_x+15, btn
                py16.rect(btn_x, btn_y, 16, 14, 0)
                py16.text(buttons[idx], btn_x + (6 if len(buttons[idx]) == 1 else 4), btn_

def calc_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    if 34 <= ly <= 106 and 6 <= lx <= 86:
        bx, by = (lx - 6) // 20, (ly - 34) // 18
        if (lx - 6) % 20 <= 16 and (ly - 34) % 18 <= 14:
            if m_held: win["pressed_btn"] = by * 4 + bx
            if m_pressed or m_sec_pressed: handle_calc_input(win, by * 4 + bx)

# -- 5. THEME (Colors & Language) ---------------------------------------------
# Einstellungen für das Erscheinungsbild des Systems.
#
# LAYOUT (von oben nach unten):
#   * DESKTOP:  16 Farben in 2 Reihen — Hintergrundfarbe des Desktops
#   * TEXT:     16 Farben — Systemtextfarbe (Menüs, Icon-Labels, Statuszeilen)
#   * CURSOR:   16 Farben — Cursor-Akzentfarbe
#   * LANGUAGE: [<] NAME [>] — Pfeile zykeln durch verfügbare Sprachen
#
# BEDIENUNG:
#   * Farbe anklicken           -> sofort übernehmen, in theme.json speichern
#   * Pfeile am Sprachschalter  -> zur nächsten/vorigen Sprache wechseln
#   * Bei nur einer Sprache (nur "EN" da) sind die Pfeile inaktiv (grau)
#
# Der CRT-Effekt ist nicht mehr im UI sichtbar, lässt sich aber durch
# manuelles Editieren von theme.json (Key: "crt_effect": true) aktivieren.
def colors_draw(win, wx, wy, ww, wh, is_active):
    py16.text(tr("DESKTOP:"), wx+6, wy+16, sys_text_color)
    for by in range(2):
        for bx in range(8):
            cid = by * 8 + bx; px, py_pos = wx + 10 + bx * 12, wy + 26 + by * 12
            py16.rectfill(px, py_pos, 10, 10, cid); py16.rect(px, py_pos, 10, 10, 0)
            if desktop_color == cid: py16.rect(px-1, py_pos-1, 12, 12, sys_text_color)

    py16.text(tr("TEXT:"), wx+6, wy+52, sys_text_color)
    for by in range(2):
        for bx in range(8):
            cid = by * 8 + bx; px, py_pos = wx + 10 + bx * 12, wy + 62 + by * 12
            py16.rectfill(px, py_pos, 10, 10, cid); py16.rect(px, py_pos, 10, 10, 0)
            if sys_text_color == cid: py16.rect(px-1, py_pos-1, 12, 12, sys_text_color)

    py16.text(tr("CURSOR:"), wx+6, wy+88, sys_text_color)
```

```
    for by in range(2):
        for bx in range(8):
            cid = by * 8 + bx; px, py_pos = wx + 10 + bx * 12, wy + 98 + by * 12
            py16.rectfill(px, py_pos, 10, 10, cid); py16.rect(px, py_pos, 10, 10, 0)
            if cursor_color == cid: py16.rect(px-1, py_pos-1, 12, 12, sys_text_color)

    # Sprachauswahl: < NAME > (statt vieler Buttons; skaliert mit beliebig vielen Sprachen
    py16.text(tr("LANGUAGE:"), wx+6, wy+124, sys_text_color)
    if _available_languages:
        idx = next((i for i, l in enumerate(_available_languages) if l["code"] == current_
        cur = _available_languages[idx]
        multi = len(_available_languages) > 1
    else:
        cur = {"code": "en", "name": "English"}; multi = False
    # Linker Pfeil
    py16.rectfill(wx+6, wy+134, 10, 10, 5 if multi else 6); py16.rect(wx+6, wy+134, 10, 10
    py16.text("<", wx+9, wy+136, 7 if multi else sys_text_color)
    # Rechter Pfeil
    arrow_r_x = wx + 100
    py16.rectfill(arrow_r_x, wy+134, 10, 10, 5 if multi else 6); py16.rect(arrow_r_x, wy+1
    py16.text(">", arrow_r_x+3, wy+136, 7 if multi else sys_text_color)
    # Name zentriert zwischen den Pfeilen
    label = str(cur.get("name", cur.get("code", "-"))).upper()
    max_chars = (arrow_r_x - (wx + 18)) // 4
    label = label[:max(1, max_chars)]
    center_x = wx + 16 + (arrow_r_x - (wx + 16)) // 2
    py16.text(label, center_x - len(label) * 2, wy+136, sys_text_color)

def _cycle_language(direction):
    """Wechselt zur Sprache direction (+1 / -1) in _available_languages."""
    if len(_available_languages) <= 1:
        return
    idx = next((i for i, l in enumerate(_available_languages) if l["code"] == current_lang
    new_idx = (idx + direction) % len(_available_languages)
    load_language(_available_languages[new_idx]["code"])
    save_theme()
    py16.tone(660, 10, py16.WAVE_SQUARE)

def colors_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    global desktop_color, sys_text_color, cursor_color
    if not (m_pressed or m_sec_pressed): return
    if 10 <= lx <= 106:
        if 26 <= ly <= 50:
            bx, by = (lx - 10) // 12, (ly - 26) // 12
            if bx < 8 and by < 2: desktop_color = by * 8 + bx; save_theme(); py16.tone(440
        elif 62 <= ly <= 86:
            bx, by = (lx - 10) // 12, (ly - 62) // 12
            if bx < 8 and by < 2: sys_text_color = by * 8 + bx; save_theme(); py16.tone(55
        elif 98 <= ly <= 122:
            bx, by = (lx - 10) // 12, (ly - 98) // 12
            if bx < 8 and by < 2: cursor_color = by * 8 + bx; save_theme(); py16.tone(330,
    # Sprach-Pfeile (lokale Fensterkoords)
    if 134 <= ly <= 144:
        if 6 <= lx <= 16:       _cycle_language(-1)
        elif 100 <= lx <= 110:  _cycle_language(+1)

# -- 6. FILES ---------------------------------------------------------------
# Dateibrowser mit Drag-&-Drop in andere App-Fenster.
#
# LAYOUT:
#   * Adresszeile oben (zeigt aktuelles Verzeichnis)
#   * Dateiliste in der Mitte (Ordner mit <DIR>-Symbol, dann Dateien)
#     Eintrag "[ .. ]" geht ins übergeordnete Verzeichnis
#   * Aktions-Schaltflächen bei aktivem Move-Modus
```

```
#
# BEDIENUNG:
#   * Eintrag anklicken -> markieren
#   * Markierter Eintrag nochmal -> Kontextmenü (OPEN, MOVE, DELETE, CANCEL)
#       - OPEN:    Ordner -> reingehen
#                  Cart (.p16/.pdf) -> startet (mit Bestätigung)
#                  Audio (.mp3/.ogg/.wav) -> spielt im MUSIC
#                  sonst -> öffnet in NOTEPAD
#       - MOVE:    aktiviert Verschieben-Modus, Klick im Zielordner setzt um
#       - DELETE:  löscht (mit modaler Bestätigung)
#   * Datei festhalten und ziehen -> Drag-&-Drop in andere Fenster:
#       - NOTEPAD  -> Text laden
#       - MUSIC    -> Audio abspielen (nur MP3/OGG/WAV)
#       - PAINT    -> .p16img-Bild in Canvas laden
#       - TERMINAL -> Pfad in die Eingabezeile setzen
#       - Desktop-Icon -> .p16img wird als App-Icon zugewiesen
#
# Verzeichnis-Typen werden in load_files() einmalig in win["dir_set"]
# gecacht — kein os.stat() pro Frame.
def load_files(win):
    try:
        raw = os.listdir(win["current_dir"])
        dirs, files = [i for i in raw if os.path.isdir(os.path.join(win["current_dir"], i)
        dirs.sort(); files.sort()
        win["items"], win["scroll"], win["selected"], win["is_sel_dir"], win["menu_open"]
        win["dir_set"] = set(dirs)   # einmal ermittelt, danach pro Frame nur Mengen-Looku
    except Exception:
        win["items"], win["selected"], win["is_sel_dir"] = ["ERROR", "NO ACCESS"], "", Fal
        win["dir_set"] = set()

def get_full_path(win, item_name): return os.path.dirname(os.path.abspath(win["current_dir

def files_draw(win, wx, wy, ww, wh, is_active):
    py16.rectfill(wx+4, wy+12, ww-8, 10, 6); py16.line(wx+4, wy+22, wx+ww-5, wy+22, 5); py
    sb_x = wx + ww - 16
    py16.rectfill(sb_x, wy+24, 10, wh-40, 6)
    py16.rectfill(sb_x, wy+14, 10, 10, 6); py16.rect(sb_x, wy+14, 10, 10, 0); py16.text("^
    py16.rectfill(sb_x, wy+wh-22, 10, 10, 6); py16.rect(sb_x, wy+wh-22, 10, 10, 0); py16.t

    visible_rows = max(1, (wh - 36) // 10)
    py16.clip(wx+4, wy+23, ww-20, wh-36)
    for i in range(visible_rows):
        idx = win["scroll"] + i
        if idx < len(win["items"]):
            item_name = win["items"][idx]; item_y = wy + 26 + (i * 10)
            text_col = 7 if win["selected"] == item_name else sys_text_color
            if win["selected"] == item_name: py16.rectfill(wx+5, item_y-2, ww-24, 10, 1)
            if is_dir_item(win, item_name):
                py16.rect(wx+7, item_y, 6, 6, text_col); py16.line(wx+7, item_y, wx+9, ite
            else: py16.rect(wx+7, item_y+1, 6, 5, text_col)
            py16.text(item_name[:20], wx+16, item_y, text_col)
    py16.clip()

    py16.line(wx+4, wy+wh-13, wx+ww-5, wy+wh-13, 5); py16.rectfill(wx+4, wy+wh-12, ww-8, 1
    if win.get("moving_file"):
        py16.text(f"MOV:{os.path.basename(win['moving_file'])[:10]}", wx+6, wy+wh-9, sys_t
        btn_p = wx + ww - 48
        py16.rectfill(btn_p, wy+wh-11, 42, 8, 5); py16.rect(btn_p, wy+wh-11, 42, 8, 0); py
        btn_c = wx + ww - 60
        py16.rectfill(btn_c, wy+wh-11, 10, 8, 5); py16.rect(btn_c, wy+wh-11, 10, 8, 0); py
    else:
        path_str = win["current_dir"]
        py16.text("..." + path_str[-17:] if len(path_str) > 20 else path_str, wx+6, wy+wh-
        if win["is_sel_dir"]:
```

```
            btn_g = wx + ww - 38
            py16.rectfill(btn_g, wy+wh-11, 32, 8, 5); py16.rect(btn_g, wy+wh-11, 32, 8, 0)

    if win.get("menu_open"):
        mx_m, my_m = wx + win["menu_x"], wy + win["menu_y"]
        py16.blend_mode("alpha", alpha=100); py16.rectfill(mx_m+2, my_m+2, 50, 48, 0); py1
        py16.rectfill(mx_m, my_m, 50, 48, 6); py16.rect(mx_m, my_m, 50, 48, 0)
        py16.line(mx_m, my_m+12, mx_m+50, my_m+12, 0); py16.line(mx_m, my_m+24, mx_m+50, m
        py16.text(tr("OPEN"), mx_m+4, my_m+4, sys_text_color); py16.text(tr("MOVE"), mx_m+

def files_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    global dragged_file
    if not (m_pressed or m_sec_pressed or m_held): return

    if win.get("menu_open"):
        if not (m_pressed or m_sec_pressed): return
        mx, my = lx - win["menu_x"], ly - win["menu_y"]
        if 0 <= mx <= 50 and 0 <= my <= 48:
            full_path = get_full_path(win, win["selected"])
            if my <= 12:
                if os.path.isdir(full_path):
                    win["current_dir"] = os.path.abspath(full_path); load_files(win); py16
                elif is_cart_file(full_path):
                    win["menu_open"] = False
                    ask_launch(full_path)
                    return
                elif full_path.lower().endswith(('.mp3', '.ogg', '.wav')):
                    music = next((w for w in windows if w["id"] == "music"), None)
                    if music:
                        if "playlist" not in music: music["playlist"] = []
                        music["playlist"].append({"name": os.path.basename(full_path), "pa
                        music["mode"], music["file_name"], music["file_path"], music["play
                        py16.music(-1)
                        try:
                            pygame.mixer.music.load(full_path); pygame.mixer.music.play()
                            music["visible"], music["minimized"] = True, False
                            windows.remove(music); windows.append(music); py16.tone(880, 2
                        except Exception: py16.tone(200, 20, py16.WAVE_SAW); music["playin
                else:
                    notepad = next((w for w in windows if w["id"] == "notepad"), None)
                    if notepad:
                        try:
                            with open(full_path, "r") as f: content = f.read().replace('\r
                            notepad["lines"] = content.split('\n') if content else [""]
                            notepad["filename"], notepad["title"] = full_path, f"NOTES [{o
                            notepad["scroll"], notepad["visible"], notepad["minimized"] =
                            windows.remove(notepad); windows.append(notepad); py16.tone(88
                        except Exception: py16.tone(200, 20, py16.WAVE_SAW)
            elif 12 < my <= 24: win["moving_file"] = full_path; py16.tone(660, 10, py16.WA
            elif 24 < my <= 36:
                if win["selected"] != "[ .. ]":
                    def _do_delete(p=full_path, w=win):
                        try:
                            os.rmdir(p) if os.path.isdir(p) else os.remove(p)
                            py16.tone(150, 25, py16.WAVE_SAW); load_files(w)
                        except Exception: py16.tone(100, 30, py16.WAVE_NOISE)
                    ask_confirm(tr("DELETE") + " " + os.path.basename(full_path)[:16] + "?
            else: py16.tone(440, 10, py16.WAVE_TRIANGLE)
        win["menu_open"] = False; return

    if win.get("moving_file"):
        if not (m_pressed or m_sec_pressed): return
        if win["w"] - 48 <= lx <= win["w"] - 6 and win["h"] - 12 <= ly <= win["h"] - 2:
            try: os.rename(win["moving_file"], os.path.join(win["current_dir"], os.path.ba
```

```
            except Exception: py16.tone(200, 20, py16.WAVE_SAW)
            win["moving_file"] = None; load_files(win); return
        elif win["w"] - 60 <= lx <= win["w"] - 50 and win["h"] - 12 <= ly <= win["h"] - 2:
            win["moving_file"] = None; py16.tone(440, 10, py16.WAVE_TRIANGLE); return

    visible_rows = max(1, (win["h"] - 36) // 10)
    if m_pressed or m_sec_pressed:
        if win["w"] - 16 <= lx <= win["w"] - 6:
            if 14 <= ly <= 24: win["scroll"] = max(0, win["scroll"] - 1); py16.tone(440, 1
            elif win["h"] - 22 <= ly <= win["h"] - 12: win["scroll"] = min(max(0, len(win[
        elif not win.get("moving_file") and win["is_sel_dir"] and win["w"] - 38 <= lx <= w
            win["current_dir"] = os.path.abspath(get_full_path(win, win["selected"])); loa

    if 6 <= lx <= win["w"] - 20 and 24 <= ly <= win["h"] - 16:
        idx = win["scroll"] + (ly - 24) // 10
        if 0 <= idx < len(win["items"]):
            clicked = win["items"][idx]
            if m_pressed or m_sec_pressed:
                if win["selected"] == clicked and clicked != "[ .. ]":
                    win["menu_open"], win["menu_x"], win["menu_y"] = True, lx, min(ly, win
                else: win["selected"] = clicked; win["is_sel_dir"] = is_dir_item(win, clic
            elif m_held and win["selected"] == clicked and clicked != "[ .. ]" and not is_
                dragged_file = get_full_path(win, clicked)

# -- 7. MUSIC PLAYER -----------------------------------------------------------
# Audio-Player für externe Dateien (MP3/OGG/WAV) und interne py-16-Tracks.
#
# LAYOUT:
#   * Anzeige: Track-Nummer (intern) oder Dateiname (extern)
#   * Steuerung: Play/Pause/Stop/Prev/Next
#   * Playlist-Bereich im Modus "external" mit Scroll
#
# BEDIENUNG:
#   * Drag-&-Drop einer Audiodatei aus FILES startet die Wiedergabe und
#     fügt sie der Playlist hinzu.
#   * Eintrag in der Playlist anklicken -> diese Stelle abspielen.
#   * Im internen Modus stehen Tracks der Cart selbst zur Verfügung
#     (Track-Nummern statt Dateinamen).
#
# Hintergrund-Tasks (update()) springen automatisch zum nächsten Playlist-
# Eintrag, wenn ein externer Track zu Ende ist (siehe _update_background_audio).
#
# Hinweis: Greift direkt auf pygame.mixer.music zu, weil py16.load_sample()
# auf 256 KB pro Datei begrenzt ist und kein Streaming bietet.
def play_playlist_idx(win, idx):
    if 0 <= idx < len(win.get("playlist", [])):
        item = win["playlist"][idx]
        win["mode"], win["file_name"], win["file_path"], win["playing"], win["list_selecte

        visible_rows = max(1, (win.get("h", 120) - 52) // 10)
        if idx < win.get("playlist_scroll", 0): win["playlist_scroll"] = idx
        elif idx >= win.get("playlist_scroll", 0) + visible_rows: win["playlist_scroll"] =

        py16.music(-1)
        try:
            pygame.mixer.music.load(item["path"]); pygame.mixer.music.play(); py16.tone(88
        except Exception: win["playing"], win["file_name"] = False, "PYGAME ERR"

def music_draw(win, wx, wy, ww, wh, is_active):
    py16.rectfill(wx+6, wy+16, ww-12, 28, 6); py16.rect(wx+6, wy+16, ww-12, 28, 0); py16.r

    if win.get("mode") == "external": py16.text(win["file_name"][:(ww-30)//4].upper(), wx+
    else: py16.text(f"TRACK {win.get('track', 0)}", wx+14, wy+22, 11 if win["playing"] els

```

```
    btn_y, bw = wy + 32, (ww - 24) // 4
    px1, px2, px3, px4 = wx+12, wx+12+bw, wx+12+2*bw, wx+12+3*bw

    py16.rectfill(px1, btn_y, bw-2, 10, 5); py16.rect(px1, btn_y, bw-2, 10, 0); py16.text(
    py16.rectfill(px2, btn_y, bw-2, 10, 5 if win["playing"] else 6); py16.rect(px2, btn_y,
    py16.rectfill(px3, btn_y, bw-2, 10, 6 if win["playing"] else 5); py16.rect(px3, btn_y,
    py16.rectfill(px4, btn_y, bw-2, 10, 5); py16.rect(px4, btn_y, bw-2, 10, 0); py16.text(

    list_y, list_h = wy + 48, wh - 52
    if list_h > 10:
        py16.rectfill(wx+6, list_y, ww-12, list_h, 0); py16.rect(wx+5, list_y-1, ww-10, li

        playlist, visible_rows, scroll, sb_x = win.get("playlist", []), max(1, list_h // 1

        py16.rectfill(sb_x, list_y, 8, list_h, 6)
        py16.rectfill(sb_x, list_y, 8, 8, 6); py16.rect(sb_x, list_y, 8, 8, 0); py16.text(
        py16.rectfill(sb_x, list_y+list_h-8, 8, 8, 6); py16.rect(sb_x, list_y+list_h-8, 8,

        py16.clip(wx+6, list_y, ww-20, list_h)
        for i in range(visible_rows):
            idx = scroll + i
            if idx < len(playlist):
                item, iy = playlist[idx], list_y + 2 + i*10
                is_playing_this = (win.get("mode") == "external" and win.get("file_path")
                color = 11 if is_playing_this else (7 if win.get("list_selected") == idx e

                if win.get("list_selected") == idx: py16.rectfill(wx+6, iy-1, ww-20, 9, 1)
                py16.text(f"{'>' if is_playing_this and win['playing'] else ' '}{item['nam
        py16.clip()

def music_update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    if not (m_pressed or m_sec_pressed): return

    bw = (win["w"] - 24) // 4
    px1, px2, px3, px4 = 12, 12+bw, 12+2*bw, 12+3*bw

    if 32 <= ly <= 42:
        if px1 <= lx <= px1+bw-2:
            if win["mode"] == "external" and win.get("playlist"):
                idx = next((i for i, x in enumerate(win["playlist"]) if x["path"] == win["
                if idx > 0: play_playlist_idx(win, idx - 1)
            else:
                win["mode"], win["track"] = "internal", max(0, win.get("track", 0) - 1)
                if win["playing"]: py16.music(win["track"])
            py16.tone(440, 10, py16.WAVE_TRIANGLE)
        elif px2 <= lx <= px2+bw-2:
            win["playing"] = True
            if win.get("mode") == "external":
                if win.get("file_path"):
                    try: pygame.mixer.music.load(win["file_path"]); pygame.mixer.music.pla
                    except Exception: win["playing"], win["file_name"] = False, "PYGAME ER
                elif win.get("playlist"): play_playlist_idx(win, 0)
            else: py16.music(win.get("track", 0))
        elif px3 <= lx <= px3+bw-2:
            win["playing"] = False; py16.music(-1)
            try: pygame.mixer.music.stop()
            except Exception: pass
        elif px4 <= lx <= px4+bw-2:
            if win["mode"] == "external" and win.get("playlist"):
                idx = next((i for i, x in enumerate(win["playlist"]) if x["path"] == win["
                if idx >= 0 and idx < len(win["playlist"]) - 1: play_playlist_idx(win, idx
            else:
                win["mode"], win["track"] = "internal", min(7, win.get("track", 0) + 1)
                if win["playing"]: py16.music(win["track"])
```

```
            py16.tone(554, 10, py16.WAVE_TRIANGLE)

    list_y, list_h = 48, win["h"] - 52
    if list_h > 10:
        sb_x, playlist, visible_rows = win["w"] - 14, win.get("playlist", []), max(1, list

        if sb_x <= lx <= sb_x + 8:
            if list_y <= ly <= list_y + 8: win["playlist_scroll"] = max(0, win.get("playli
            elif list_y + list_h - 8 <= ly <= list_y + list_h: win["playlist_scroll"] = mi
        elif 6 <= lx <= sb_x - 2 and list_y <= ly <= list_y + list_h:
            idx = win.get("playlist_scroll", 0) + (ly - list_y) // 10
            if 0 <= idx < len(playlist):
                if win.get("list_selected") == idx: play_playlist_idx(win, idx)
                else: win["list_selected"] = idx; py16.tone(880, 10, py16.WAVE_SQUARE)

# --- DIE ZENTRALE REGISTRY ---
APPS = {
    "notepad": {"draw": notepad_draw, "update": notepad_update},
    "files": {"draw": files_draw, "update": files_update},
    "colors": {"draw": colors_draw, "update": colors_update},
    "calc": {"draw": calc_draw, "update": calc_update},
    "paint": {"draw": paint_draw, "update": paint_update},
    "terminal": {"draw": terminal_draw, "update": terminal_update},
    "music": {"draw": music_draw, "update": music_update}
}

# ========================================
# ====== PLUGIN-SYSTEM (Ordner "apps/") ====
# ========================================
# Lege .py-Dateien in den Ordner "apps/" neben der Cart. Jede Datei ist
# eine Mini-App mit dieser Schnittstelle:
#
#   APP = {"id": "myapp", "name": "MYAPP", "w": 120, "h": 90,
#          "resizable": True, "min_w": 80, "min_h": 60}
#   def init(win): ...                                  # optional
#   def update(win, lx, ly, m_pressed, m_sec_pressed, m_held): ...
#   def draw(win, wx, wy, ww, wh, is_active): ...
#
# Die OS zeichnet Rahmen/Titelleiste/Buttons selbst; das Plugin zeichnet
# nur den Fensterinhalt (wx,wy = linke obere Fensterecke).

PLUGIN_DIR = "apps"
PLUGIN_APPS = []          # [{"id","name"}] fuer das Desktop-"ADD"-Menue
_plugin_icon_path = {}    # app_id -> Pfad zur .p16img (aus APP["icon"])
_loaded_plugin_files = set()

_EXAMPLE_PLUGIN = '''# Beispiel-Plugin. Frei kopieren/aendern. Eine Datei = eine App.
# Optional kann ein .p16img-Bild als App-Icon angegeben werden (Pfad
# relativ zum apps/-Ordner). Das Bild wird in PAINT gezeichnet, mit SAV
# als img_NNN.p16img gespeichert und z.B. nach apps/ kopiert.
APP = {"id": "ticker", "name": "TICKER", "w": 120, "h": 72, "resizable": False,
       # "icon": "ticker.p16img",
      }

def init(win):
    win["n"] = 0

def update(win, lx, ly, m_pressed, m_sec_pressed, m_held):
    if m_pressed:
        win["n"] = win.get("n", 0) + 1

def draw(win, wx, wy, ww, wh, is_active):
    import py16
    py16.text("HELLO FROM PLUGIN", wx + 6, wy + 18, 1)
```

```
    py16.text("CLICKS: " + str(win.get("n", 0)), wx + 6, wy + 32, 8)
    py16.text("EDIT apps/example.py", wx + 6, wy + 48, 5)
'''

def _make_safe(fn, label):
    """Kapselt Plugin-Funktionen, damit ein Fehler nicht die ganze OS abstuerzt."""
    def wrapper(*a, **k):
        try:
            return fn(*a, **k)
        except Exception as e:
            if label == "draw" and len(a) >= 5:
                wx, wy, ww = a[1], a[2], a[3]
                try:
                    py16.rectfill(wx + 4, wy + 14, ww - 8, 22, 8)
                    py16.text("PLUGIN ERROR", wx + 8, wy + 18, 7)
                    py16.text(str(e)[:22], wx + 8, wy + 27, 7)
                except Exception:
                    pass
    return wrapper

def _register_plugin(mod):
    meta = getattr(mod, "APP", None)
    if not isinstance(meta, dict) or "id" not in meta:
        return False, "NO APP DICT"
    pid = str(meta["id"])
    if pid in APPS:
        return False, "ID BELEGT"
    if not (hasattr(mod, "draw") and hasattr(mod, "update")):
        return False, "DRAW/UPDATE FEHLT"

    name = str(meta.get("name", pid)).upper()[:10]
    idx = len(PLUGIN_APPS)
    win = {
        "id": pid, "title": name + ".PY16",
        "x": 30 + (idx * 12) % 80, "y": 24 + (idx * 10) % 60,
        "w": int(meta.get("w", 140)), "h": int(meta.get("h", 110)),
        "visible": False, "minimized": False,
        "resizable": bool(meta.get("resizable", False)),
        "min_w": int(meta.get("min_w", 60)),
        "min_h": int(meta.get("min_h", 50)),
        "_plugin": True,
    }
    if hasattr(mod, "init"):
        try: mod.init(win)
        except Exception: pass

    APPS[pid] = {"draw": _make_safe(mod.draw, "draw"),
                 "update": _make_safe(mod.update, "update")}
    windows.append(win)
    PLUGIN_APPS.append({"id": pid, "name": name})
    # Icon-Aufloesung in Prioritaet:
    #   1. explizit:    APP["icon"]
    #   2. Konvention:  apps/<plugin-dateiname>.p16img (gleicher Stamm wie .py)
    #   3. Konvention:  apps/<app-id>.p16img (siehe _apply_default_icons fuer alle Apps)
    icon_rel = meta.get("icon")
    if isinstance(icon_rel, str) and icon_rel:
        icon_abs = icon_rel if os.path.isabs(icon_rel) else os.path.join(PLUGIN_DIR, icon_
        _plugin_icon_path[pid] = icon_abs
    else:
        plugin_file = getattr(mod, "__file__", None)
        if plugin_file:
            stem = os.path.splitext(os.path.basename(plugin_file))[0]
            cand = os.path.join(PLUGIN_DIR, stem + ".p16img")
            if os.path.isfile(cand):
```

```
                _plugin_icon_path[pid] = cand
    # Auf dem Desktop ablegen NUR wenn dieses Plugin noch nie gesehen wurde
    # ("Erstkontakt"). So bleiben spaetere User-Entscheidungen erhalten:
    # ein per Rechtsklick geloeschtes Icon kommt nicht bei jedem Start zurueck.
    is_first_time = pid not in known_plugins
    if is_first_time:
        known_plugins.append(pid)
        if not any(i["id"] == pid for i in desktop_icons):
            desktop_icons.append({"id": pid, "name": name})
        save_theme()
    return True, name

def _apply_default_icons():
    """Konvention: apps/<app-id>.p16img -> Icon fuer App mit dieser ID
    (gilt fuer eingebaute Apps und Plugins). Ueberschreibt nichts, was
    explizit per APP["icon"] gesetzt wurde oder bereits per Konvention
    aus dem Plugin-Stamm-Namen aufgeloest ist."""
    if not os.path.isdir(PLUGIN_DIR):
        return 0
    try:
        files_in_apps = set(n.lower() for n in os.listdir(PLUGIN_DIR) if n.lower().endswit
    except Exception:
        return 0
    added = 0
    for app_id in list(APPS.keys()):
        if app_id in _plugin_icon_path:
            continue
        cand_name = app_id + ".p16img"
        if cand_name.lower() in files_in_apps:
            # exakten Dateinamen mit Originalschreibweise rekonstruieren
            real = next((n for n in os.listdir(PLUGIN_DIR) if n.lower() == cand_name.lower
            _plugin_icon_path[app_id] = os.path.join(PLUGIN_DIR, real)
            _image_cache.pop(os.path.abspath(_plugin_icon_path[app_id]), None)
            added += 1
    return added

def load_plugins():
    """Scannt apps/ und registriert NEUE .py-Dateien. Liefert Statuszeilen."""
    log = []
    try:
        if not os.path.isdir(PLUGIN_DIR):
            os.makedirs(PLUGIN_DIR, exist_ok=True)
            with open(os.path.join(PLUGIN_DIR, "example.py"), "w") as f:
                f.write(_EXAMPLE_PLUGIN)
        files = sorted(n for n in os.listdir(PLUGIN_DIR) if n.endswith(".py"))
    except Exception:
        return ["APPS DIR ERROR"]
    for fn in files:
        path = os.path.abspath(os.path.join(PLUGIN_DIR, fn))
        if path in _loaded_plugin_files:
            continue
        _loaded_plugin_files.add(path)
        try:
            modname = "plugin_" + os.path.splitext(fn)[0]
            spec = importlib.util.spec_from_file_location(modname, path)
            mod = importlib.util.module_from_spec(spec)
            sys.modules[modname] = mod
            spec.loader.exec_module(mod)
            ok, info = _register_plugin(mod)
            log.append(("+ " if ok else "! ") + fn[:14] + " " + info)
        except Exception as e:
            log.append("! " + fn[:14] + " " + str(e)[:18])
    added = _apply_default_icons()
    if added:
```

```
1473          log.append("+ " + str(added) + " ICON(S) FROM apps/")
1474      if not log:
1475          log.append("NO NEW PLUGINS")
1476      return log
1477
1478  # ========================================
1479  # ====== SYSTEM KERNEL (MAIN LOOPS) ========
1480  # ========================================
1481
1482  def init():
1483      """Startup-Hook. Wird einmal beim Cart-Start aufgerufen.
1484
1485      Reihenfolge ist wichtig:
1486        1. load_theme()        -> Farben, desktop_icons, known_plugins, language
1487        2. discover_languages  -> scannt lang/, legt beim Erststart de.json an
1488        3. load_language       -> aktiviert die in theme.json gespeicherte Sprache
1489        4. Window-Init         -> FILES und TERMINAL kriegen ihr Arbeitsverzeichnis
1490        5. load_plugins        -> apps/ scannen, neue Plugins registrieren
1491      """
1492      load_theme()
1493      discover_languages()
1494      load_language(current_language)
1495      for w in windows:
1496          if w["id"] == "files": w["current_dir"] = os.path.abspath("."); load_files(w)
1497          if w["id"] == "terminal": w["cwd"] = os.path.abspath(".")
1498      load_plugins()
1499
1500  def _update_background_audio():
1501      """Spielt bei externem Modus automatisch den naechsten Track der Playlist."""
1502      music_win = next((w for w in windows if w["id"] == "music"), None)
1503      if music_win and music_win.get("playing") and music_win.get("mode") == "external":
1504          try:
1505              if not pygame.mixer.music.get_busy():
1506                  playlist, idx = music_win.get("playlist", []), music_win.get("list_selecte
1507                  if playlist and 0 <= idx < len(playlist) - 1: play_playlist_idx(music_win,
1508                  else: music_win["playing"] = False
1509          except Exception: pass
1510
1511  def _update_cursor():
1512      """Vereinheitlichte Maus-+-Gamepad-Cursorbewegung."""
1513      global cursor_x, cursor_y, prev_mx, prev_my
1514      cur_mx, cur_my = py16.mouse_x(), py16.mouse_y()
1515      if cur_mx != prev_mx or cur_my != prev_my:
1516          cursor_x, cursor_y = cur_mx, cur_my
1517          prev_mx, prev_my = cur_mx, cur_my
1518      if py16.btn('left'): cursor_x -= 2
1519      if py16.btn('right'): cursor_x += 2
1520      if py16.btn('up'): cursor_y -= 2
1521      if py16.btn('down'): cursor_y += 2
1522      cursor_x = max(0, min(py16.WIDTH-1, cursor_x))
1523      cursor_y = max(0, min(py16.HEIGHT-1, cursor_y))
1524
1525  # Feste Geometrie des Bestätigungsdialogs (von Draw + Hit-Test gemeinsam genutzt)
1526  _CONFIRM_W, _CONFIRM_H = 130, 46
1527  def _confirm_geometry():
1528      bx = (py16.WIDTH - _CONFIRM_W) // 2
1529      by = (py16.HEIGHT - _CONFIRM_H) // 2
1530      yes_x = bx + _CONFIRM_W - 70
1531      no_x = bx + _CONFIRM_W - 34
1532      btn_y = by + _CONFIRM_H - 14
1533      return bx, by, yes_x, no_x, btn_y
1534
1535  def _handle_confirm_dialog(mx, my, m_pressed, m_sec_pressed):
1536      """Modaler Dialog: faengt ALLE Klicks ab. True = Eingabe verbraucht."""
```

```
    global confirm_dialog
    if confirm_dialog is None:
        return False
    if m_pressed or m_sec_pressed:
        _bx, _by, yes_x, no_x, btn_y = _confirm_geometry()
        if btn_y <= my <= btn_y + 10:
            if yes_x <= mx <= yes_x + 30:
                cb = confirm_dialog["on_yes"]; confirm_dialog = None
                cb(); py16.tone(660, 15, py16.WAVE_SQUARE)
            elif no_x <= mx <= no_x + 28:
                confirm_dialog = None; py16.tone(440, 10, py16.WAVE_TRIANGLE)
    return True

def draw_confirm():
    """Zeichnet den modalen Bestätigungsdialog samt Abdunklung."""
    if confirm_dialog is None:
        return
    bx, by, yes_x, no_x, btn_y = _confirm_geometry()
    py16.blend_mode("alpha", alpha=120)
    py16.rectfill(0, 0, py16.WIDTH, py16.HEIGHT, 0)
    py16.blend_mode("normal")
    draw_win_border(bx, by, _CONFIRM_W, _CONFIRM_H)
    txt = confirm_dialog["text"]
    py16.text(txt[:30], bx + 6, by + 8, sys_text_color)
    if len(txt) > 30:
        py16.text(txt[30:60], bx + 6, by + 16, sys_text_color)
    py16.rectfill(yes_x, btn_y, 30, 10, 5); py16.rect(yes_x, btn_y, 30, 10, 0)
    py16.text(tr("YES"), yes_x + 7, btn_y + 2, 7)
    py16.rectfill(no_x, btn_y, 28, 10, 6); py16.rect(no_x, btn_y, 28, 10, 0)
    py16.text(tr("NO"), no_x + 8, btn_y + 2, sys_text_color)

def _handle_desktop_click(mx, my, m_pressed, m_sec_pressed):
    """Klicks auf Desktop-Icons / leeren Desktop (war das Ende von update())."""
    global selected_desktop_icon, context_menu_open, context_menu_options
    global context_menu_x, context_menu_y
    icon_clicked = False
    for i, icon in enumerate(desktop_icons):
        ix, iy = 8 + (i // 5) * 40, 8 + (i % 5) * 36
        if ix <= mx <= ix + 24 and iy <= my <= iy + 24:
            icon_clicked = True
            if m_pressed:
                if selected_desktop_icon == icon["id"]:
                    if icon.get("is_file"):
                        if is_cart_file(icon["id"]):
                            selected_desktop_icon = None
                            ask_launch(icon["id"])
                            break
                        notepad = next((w for w in windows if w["id"] == "notepad"), None)
                        if notepad:
                            try:
                                with open(icon["id"], "r") as f: content = f.read().replac
                                notepad["lines"] = content.split('\n') if content else [""
                                notepad["filename"], notepad["title"] = icon["id"], f"NOTE
                                notepad["scroll"], notepad["visible"], notepad["minimized""
                                windows.remove(notepad); windows.append(notepad); py16.ton
                            except Exception: pass
                    else:
                        for w in windows:
                            if w["id"] == icon["id"]:
                                w["visible"], w["minimized"] = True, False
                                windows.remove(w); windows.append(w); py16.tone(880, 10, p
                                break
                    selected_desktop_icon = None
                else:
```

```
                    selected_desktop_icon = icon["id"]
                    py16.tone(440, 10, py16.WAVE_TRIANGLE)
            elif m_sec_pressed:
                selected_desktop_icon = icon["id"]
                context_menu_open = True
                context_menu_options = ["DELETE", "CANCEL"]
                context_menu_x = mx
                menu_h = len(context_menu_options) * 12 + 2
                context_menu_y = min(my, py16.HEIGHT - menu_h - 12)
                py16.tone(440, 10, py16.WAVE_TRIANGLE)
            break
    if not icon_clicked:
        if m_pressed:
            selected_desktop_icon = None
        elif m_sec_pressed:
            selected_desktop_icon = None
            context_menu_open = True
            # Dynamische Optionen aufbauen: Nur fehlende Apps anzeigen
            context_menu_options = ["NEW TXT"]
            for app in DEFAULT_APPS + PLUGIN_APPS:
                if not any(i["id"] == app["id"] for i in desktop_icons):
                    context_menu_options.append("ADD " + app["name"])
            context_menu_options.append("CANCEL")
            context_menu_x = mx
            menu_h = len(context_menu_options) * 12 + 2
            context_menu_y = min(my, py16.HEIGHT - menu_h - 12)
            py16.tone(440, 10, py16.WAVE_TRIANGLE)

def update():
    """Hauptschleife für Eingaben und Logik. Wird 60x pro Sekunde von py16 aufgerufen.

    Verarbeitungs-Reihenfolge (Priorität von hoch nach niedrig):
      1. Background-Tasks (Audio-Autoplay)
      2. Cursor-Position aus Maus + Gamepad zusammenrechnen
      3. Modaler Bestätigungsdialog hat Vorrang vor allem
      4. Start-Menü und Datums-Popup
      5. Kontextmenüs (FILES, Desktop, Desktop-Icons)
      6. Aktives Fenster: Titelleiste / Schließen / Min / Max / Resize / Drag
      7. App-spezifisches *_update mit lokalen Koordinaten (lx, ly)
      8. Klicks auf den Desktop / Desktop-Icons
    """
    global dragged_window, drag_off_x, drag_off_y, start_menu_open, date_popup_open
    global resized_window, resize_start_w, resize_start_h, resize_start_mx, resize_start_m
    global dragged_file, selected_desktop_icon, context_menu_open

    _update_background_audio()
    _update_cursor()

    m_pressed = py16.mouse_btnp(0) or py16.btnp('z') or py16.btnp('space') or py16.btnp('e
    m_held = py16.mouse_btn(0) or py16.btn('z') or py16.btn('space') or py16.btn('enter')
    m_sec_pressed = py16.mouse_btnp(2) or py16.btnp('x')

    mx, my = int(cursor_x), int(cursor_y)
    for w in windows:
        if w["id"] == "calc": w["pressed_btn"] = -1

    # Modaler Bestätigungsdialog hat Vorrang vor allem anderen
    if _handle_confirm_dialog(mx, my, m_pressed, m_sec_pressed):
        return


    if dragged_file is not None:
        if not m_held:
            drop_win = None
```

```
            for i in range(len(windows)-1, -1, -1):
                win = windows[i]
                if win["visible"] and not win["minimized"] and win["x"] <= mx <= win["x"]
                    drop_win = win; break
            if drop_win:
                full_path = dragged_file
                if drop_win["id"] == "notepad" and not os.path.isdir(full_path):
                    try:
                        with open(full_path, "r") as f: content = f.read().replace('\r', '
                        drop_win["lines"] = content.split('\n') if content else [""]
                        drop_win["filename"], drop_win["title"], drop_win["scroll"] = full
                        drop_win["visible"], drop_win["minimized"] = True, False
                        windows.remove(drop_win); windows.append(drop_win); py16.tone(880,
                    except Exception: py16.tone(200, 20, py16.WAVE_SAW)
                elif drop_win["id"] == "music" and full_path.lower().endswith(('.mp3', '.o
                    if "playlist" not in drop_win: drop_win["playlist"] = []
                    drop_win["playlist"].append({"name": os.path.basename(full_path), "pat
                    drop_win["mode"], drop_win["file_name"], drop_win["file_path"], drop_w
                    drop_win["visible"], drop_win["minimized"] = True, False
                    windows.remove(drop_win); windows.append(drop_win); py16.music(-1)
                    try:
                        pygame.mixer.music.load(full_path); pygame.mixer.music.play(); py1
                    except Exception: py16.tone(200, 20, py16.WAVE_SAW); drop_win["playing
                elif drop_win["id"] == "terminal":
                    drop_win["input_str"] = full_path
                    drop_win["visible"], drop_win["minimized"] = True, False
                    windows.remove(drop_win); windows.append(drop_win); py16.tone(660, 10,
                elif drop_win["id"] == "paint" and full_path.lower().endswith(".p16img"):
                    px = load_p16img(full_path)
                    if px is not None:
                        drop_win["canvas"] = list(px)
                        drop_win["status"] = "LOADED " + os.path.basename(full_path)[:14]
                        drop_win["visible"], drop_win["minimized"] = True, False
                        windows.remove(drop_win); windows.append(drop_win); py16.tone(880,
                    else:
                        drop_win["status"] = "LOAD FAILED"
                        py16.tone(200, 20, py16.WAVE_SAW)
            else:
                # Kein Fenster getroffen: Drop auf Desktop-Icon? (.p16img -> Icon zuweisen
                if full_path.lower().endswith(".p16img"):
                    for i, ic in enumerate(desktop_icons):
                        ix, iy = 8 + (i // 5) * 40, 8 + (i % 5) * 36
                        if ix <= mx <= ix + 24 and iy <= my <= iy + 24:
                            ic["icon_image"] = full_path
                            _image_cache.pop(os.path.abspath(full_path), None)   # frisch
                            save_theme()
                            py16.tone(880, 20, py16.WAVE_SQUARE)
                            break
                dragged_file = None
        return

    if resized_window is not None:
        if m_held:
            resized_window["w"] = max(resized_window.get("min_w", 60), resize_start_w + (m
            resized_window["h"] = max(resized_window.get("min_h", 60), resize_start_h + (m
        else: resized_window = None
    elif dragged_window is not None:
        if m_held:
            dragged_window["x"] = mx - drag_off_x
            dragged_window["y"] = my - drag_off_y
        else: dragged_window = None
    elif m_held:
        for i in range(len(windows)-1, -1, -1):
            win = windows[i]
```

```
            if not win["visible"] or win["minimized"]: continue
            if win["x"] <= mx <= win["x"] + win["w"] and win["y"] <= my <= win["y"] + win[
                if win["id"] in APPS: APPS[win["id"]]["update"](win, mx - win["x"], my - w
                break

    if m_pressed or m_sec_pressed:
        # Taskbar Logic
        if py16.HEIGHT-12 <= my <= py16.HEIGHT:
            if start_menu_open or context_menu_open:
                start_menu_open, context_menu_open = False, False
                return

            # Start-Button Click
            if 2 <= mx <= 62:
                start_menu_open = True
                py16.tone(440, 10, py16.WAVE_SQUARE)
                return

            # Minimized Windows Click
            minimized_wins = [w for w in windows if w["visible"] and w["minimized"]]
            for i, w in enumerate(minimized_wins):
                bx = 68 + i * 46
                if bx <= mx <= bx + 44:
                    w["minimized"] = False
                    windows.remove(w); windows.append(w); py16.tone(660, 10, py16.WAVE_SQU
                    return

            # Clock Click
            clock_x = py16.WIDTH - 44
            if clock_x <= mx <= py16.WIDTH:
                date_popup_open = not date_popup_open
                py16.tone(880, 10, py16.WAVE_SQUARE)
                return

            return # Blockiere andere Fenster-Clicks, falls leere Taskleiste getroffen

        # Menüs schließen, falls irgendwo anders geklickt wurde
        if context_menu_open:
            menu_h = len(context_menu_options) * 12 + 2
            if context_menu_x <= mx <= context_menu_x + 56 and context_menu_y <= my <= con
                if m_pressed:
                    idx = (my - context_menu_y - 2) // 12
                    if 0 <= idx < len(context_menu_options):
                        opt = context_menu_options[idx]
                        if opt == "DELETE":
                            if selected_desktop_icon:
                                icon_to_del = next((i for i in desktop_icons if i["id"] ==
                                if icon_to_del:
                                    def _do_delete_icon(ic=icon_to_del):
                                        global selected_desktop_icon
                                        if ic.get("is_file"):
                                            try: os.remove(ic["id"])
                                            except Exception: pass
                                        if ic in desktop_icons: desktop_icons.remove(ic)
                                        save_theme()
                                        py16.tone(150, 25, py16.WAVE_SAW) # Löschen-Sound
                                        selected_desktop_icon = None
                                    ask_confirm(tr("DELETE") + " " + str(icon_to_del["name
                        elif opt == "NEW TXT":
                            i = 1
                            while os.path.exists(f"note{i}.txt"): i += 1
                            new_file = f"note{i}.txt"
                            try:
                                with open(new_file, "w") as f: f.write("")
```

```
                                desktop_icons.append({"id": new_file, "name": new_file, "i
                                save_theme()
                                py16.tone(880, 10, py16.WAVE_SQUARE)
                            except Exception: pass
                        elif opt.startswith("ADD "):
                            app_name = opt[4:]
                            app_info = next((a for a in DEFAULT_APPS + PLUGIN_APPS if a["n
                            if app_info:
                                desktop_icons.append({"id": app_info["id"], "name": app_in
                                save_theme()
                                py16.tone(880, 10, py16.WAVE_SQUARE)
                        else: # CANCEL
                            py16.tone(440, 10, py16.WAVE_TRIANGLE)
                context_menu_open = False
                return # Klick auf das Menü abfangen
            else:
                context_menu_open = False # Schließen und Klick durchlassen

        if start_menu_open or date_popup_open:
            if start_menu_open:
                menu_y = py16.HEIGHT - 12 - (len(windows) * 12) - 4
                if 0 <= mx <= 80 and menu_y <= my <= py16.HEIGHT-12:
                    click_idx = (my - menu_y - 2) // 12
                    if 0 <= click_idx < len(windows):
                        win = windows.pop(click_idx)
                        win["visible"], win["minimized"] = True, False
                        windows.append(win); py16.tone(880, 10, py16.WAVE_SQUARE)
            start_menu_open, date_popup_open = False, False

        # Window Hit Detection
        window_hit = False
        for i in range(len(windows)-1, -1, -1):
            win = windows[i]
            if not win["visible"] or win["minimized"]: continue

            if win["x"] <= mx <= win["x"] + win["w"] and win["y"] <= my <= win["y"] + win[
                window_hit = True
                selected_desktop_icon = None
                windows.append(windows.pop(i)) # Bring to front
                lx, ly = mx - win["x"], my - win["y"]

                if m_sec_pressed: break

                if 5 <= lx <= 12 and 4 <= ly <= 11: win["visible"] = False
                elif win["w"] - 14 <= lx <= win["w"] - 7 and 4 <= ly <= 11: win["minimized
                elif win.get("resizable", False) and win["w"] - 24 <= lx <= win["w"] - 17
                    if not win.get("maximized"):
                        win["prev_x"], win["prev_y"], win["prev_w"], win["prev_h"] = win["
                        win["x"], win["y"], win["w"], win["h"], win["maximized"] = 0, 0, p
                        py16.tone(660, 15, py16.WAVE_SQUARE)
                    else:
                        win["x"], win["y"], win["w"], win["h"], win["maximized"] = win.get
                        py16.tone(440, 15, py16.WAVE_SQUARE)
                elif ly <= 12 and not win.get("maximized", False):
                    dragged_window, drag_off_x, drag_off_y = win, lx, ly
                elif win.get("resizable", False) and not win.get("maximized", False) and l
                    resized_window, resize_start_w, resize_start_h, resize_start_mx, resiz
                elif win["id"] in APPS:
                    APPS[win["id"]]["update"](win, lx, ly, m_pressed, m_sec_pressed, False
                break

        if not window_hit and (m_pressed or m_sec_pressed):
            _handle_desktop_click(mx, my, m_pressed, m_sec_pressed)

```

```
def draw_win_border(x, y, w, h):
    py16.rectfill(x, y, w, h, 0); py16.rectfill(x+1, y+1, w-2, h-2, COLOR_BORDER_LT)
    py16.rectfill(x+2, y+2, w-4, h-4, 6)
    py16.line(x+1, y+h-2, x+w-2, y+h-2, COLOR_BORDER_DK); py16.line(x+w-2, y+1, x+w-2, y+h

def draw_window_shadow(win):
    py16.blend_mode("alpha", alpha=80)
    py16.rectfill(win["x"] + 6, win["y"] + 6, win["w"], win["h"], 0)
    py16.blend_mode("normal")

def draw_window(win, is_active):
    wx, wy, ww, wh = win["x"], win["y"], win["w"], win["h"]
    draw_win_border(wx, wy, ww, wh); py16.rectfill(wx+4, wy+12, ww-8, wh-16, 7)

    # OS Title Bar
    py16.rectfill(wx+3, wy+3, ww-6, 9, COLOR_TITLE_BAR if is_active else 5)
    py16.text(win["title"], wx+16, wy+5, 7 if is_active else 6)

    # Controls
    py16.rectfill(wx+5, wy+4, 7, 7, 6); py16.rect(wx+5, wy+4, 7, 7, 0); py16.line(wx+6, wy
    min_x = wx + ww - 14
    py16.rectfill(min_x, wy+4, 7, 7, 6); py16.rect(min_x, wy+4, 7, 7, 0); py16.line(min_x+

    if win.get("resizable", False):
        max_x = wx + ww - 24
        py16.rectfill(max_x, wy+4, 7, 7, 6); py16.rect(max_x, wy+4, 7, 7, 0)
        if win.get("maximized"):
            py16.rect(max_x+1, wy+7, 3, 3, 0); py16.rect(max_x+3, wy+5, 3, 3, 0)
        else:
            py16.rect(max_x+1, wy+5, 5, 5, 0); py16.line(max_x+1, wy+6, max_x+5, wy+6, 0)

    if win.get("resizable", False) and not win.get("minimized", False) and not win.get("ma
        rx, ry = wx + ww - 8, wy + wh - 8
        py16.line(rx+6, ry, rx, ry+6, 5); py16.line(rx+6, ry+3, rx+3, ry+6, 5)

    if win["id"] in APPS: APPS[win["id"]]["draw"](win, wx, wy, ww, wh, is_active)

def draw():
    """Frame-Render-Funktion. Wird 60x pro Sekunde NACH update() aufgerufen.

    Z-Order von unten nach oben (alles spätere überdeckt alles frühere):
      1. Hintergrundfarbe (desktop_color)
      2. Desktop-Icons (mit eigenem .p16img oder Default-Symbol)
      3. Sichtbare Fenster (älteste zuerst, aktives zuletzt)
      4. Taskleiste mit minimierten Fenstern und Datums-Anzeige
      5. Start-Menü und Datums-Popup (falls geöffnet)
      6. Kontextmenüs
      7. Modaler Bestätigungsdialog
      8. Drag-Indikator (wenn eine Datei mit der Maus gezogen wird)
      9. Cursor (immer ganz oben sichtbar)
     10. CRT-Effekt-Overlay (falls in theme.json aktiviert)
    """
    py16.cls(desktop_color)

    # --- DESKTOP ICONS ---
    for i, icon in enumerate(desktop_icons):
        ix, iy = 8 + (i // 5) * 40, 8 + (i % 5) * 36
        app_id = icon["id"]

        if app_id == selected_desktop_icon:
            py16.blend_mode("alpha", alpha=100)
            py16.rectfill(ix - 4, iy - 2, 28, 28, 1)
            py16.blend_mode("normal")

```

```
        # Pfad zum eigenen Bild: explizit gesetztes icon_image schlaegt
        # das Plugin-Default-Icon (App.icon im Plugin-Dict).
        img_path = icon.get("icon_image")
        if not img_path:
            plug_icon = _plugin_icon_path.get(app_id)
            if plug_icon: img_path = plug_icon
        custom_drawn = False
        if img_path and os.path.isfile(img_path):
            custom_drawn = draw_icon_image(img_path, ix, iy)

        if custom_drawn:
            pass
        elif icon.get("is_file"):
            py16.rectfill(ix+4, iy, 12, 14, 7); py16.rect(ix+4, iy, 12, 14, 0); py16.line(
        elif app_id == "files":
            py16.rectfill(ix+2, iy+4, 16, 10, 10); py16.rectfill(ix+2, iy+2, 6, 2, 10); py
        elif app_id == "notepad":
            py16.rectfill(ix+4, iy, 12, 14, 7); py16.rect(ix+4, iy, 12, 14, 0); py16.line(
        elif app_id == "terminal":
            py16.rectfill(ix+2, iy+2, 16, 12, 0); py16.rect(ix+2, iy+2, 16, 12, 5); py16.t
        elif app_id == "paint":
            py16.circfill(ix+10, iy+8, 6, 6); py16.circ(ix+10, iy+8, 6, 0); py16.circfill(
        elif app_id == "calc":
            py16.rectfill(ix+4, iy, 12, 14, 6); py16.rect(ix+4, iy, 12, 14, 0); py16.rectf
            py16.pset(ix+6, iy+7, 0); py16.pset(ix+9, iy+7, 0); py16.pset(ix+12, iy+7, 0);
        elif app_id == "music":
            py16.rectfill(ix+2, iy+2, 16, 12, 1); py16.rect(ix+2, iy+2, 16, 12, 0); py16.c
        elif app_id == "colors":
            py16.rectfill(ix+2, iy+2, 16, 12, 7); py16.rect(ix+2, iy+2, 16, 12, 0)
            py16.rectfill(ix+4, iy+4, 4, 4, 8); py16.rectfill(ix+8, iy+4, 4, 4, 11); py16.
            py16.rectfill(ix+4, iy+8, 4, 4, 10); py16.rectfill(ix+8, iy+8, 4, 4, 14); py16
        else:
            # Generisches Icon fuer Plugin-Apps
            py16.rectfill(ix+3, iy+2, 14, 12, 12); py16.rect(ix+3, iy+2, 14, 12, 0)
            py16.rectfill(ix+6, iy+5, 8, 6, 7); py16.pset(ix+10, iy+8, 1)
            py16.line(ix+3, iy+2, ix+16, iy+2, 7)

        tw = len(icon["name"]) * 4
        text_x = ix + 10 - tw // 2
        py16.text(icon["name"], text_x, iy + 18, 0)
        py16.text(icon["name"], text_x, iy + 17, sys_text_color)

    for i, win in enumerate(windows):
        if win["visible"] and not win["minimized"]:
            draw_window_shadow(win); draw_window(win, i == len(windows) - 1)

    # Taskbar & Start Menu
    py16.rectfill(0, py16.HEIGHT-12, py16.WIDTH, 12, 6); py16.line(0, py16.HEIGHT-12, py16
    py16.rectfill(2, py16.HEIGHT-11, 60, 10, 5 if start_menu_open else 6); py16.rect(2, py

    minimized_wins = [w for w in windows if w["visible"] and w["minimized"]]
    for i, w in enumerate(minimized_wins):
        bx, by = 68 + i * 46, py16.HEIGHT - 11
        py16.rectfill(bx, by, 44, 10, 6)
        py16.line(bx, by, bx+43, by, 15); py16.line(bx, by, bx, by+9, 15); py16.line(bx, b
        py16.rect(bx, by, 44, 10, 0); py16.text(w["title"][:6], bx + 4, by + 3, sys_text_c

    t = time.localtime()
    clock_x = py16.WIDTH - 44
    py16.rectfill(clock_x - 4, py16.HEIGHT-11, 46, 10, 5); py16.rectfill(clock_x - 3, py16
    py16.line(clock_x - 3, py16.HEIGHT-2, clock_x + 40, py16.HEIGHT-2, 15); py16.line(cloc
    py16.text(f"{t.tm_hour:02d}:{t.tm_min:02d}:{t.tm_sec:02d}", clock_x, py16.HEIGHT-7 if

    if start_menu_open:
```

```
        menu_h = (len(windows) * 12) + 4
        menu_y = py16.HEIGHT - 12 - menu_h
        draw_win_border(0, menu_y, 80, menu_h)
        for i, win in enumerate(windows):
            item_y = menu_y + 2 + (i * 12)
            if win["visible"]: py16.pset(6, item_y + 3, sys_text_color); py16.pset(7, item
            py16.text(win["title"], 12, item_y + 4, sys_text_color)

    if date_popup_open:
        days = ["MO", "DI", "MI", "DO", "FR", "SA", "SO"]
        date_str = f"{days[t.tm_wday]}, {t.tm_mday:02d}.{t.tm_mon:02d}.{t.tm_year}"
        box_w = len(date_str) * 4 + 8
        box_x, box_y = py16.WIDTH - box_w - 2, py16.HEIGHT - 26
        draw_win_border(box_x, box_y, box_w, 12); py16.text(date_str, box_x + 4, box_y + 4

    mx, my = int(cursor_x), int(cursor_y)

    # --- KONTEXTMENÜ ZEICHNEN ---
    if context_menu_open:
        mx_m, my_m = context_menu_x, context_menu_y
        menu_h = len(context_menu_options) * 12 + 2
        py16.blend_mode("alpha", alpha=100); py16.rectfill(mx_m+2, my_m+2, 56, menu_h, 0);
        py16.rectfill(mx_m, my_m, 56, menu_h, 6); py16.rect(mx_m, my_m, 56, menu_h, 0)

        for i, opt in enumerate(context_menu_options):
            py16.text(context_label(opt), mx_m+4, my_m + 4 + i*12, sys_text_color)
            if i < len(context_menu_options) - 1:
                py16.line(mx_m, my_m + 13 + i*12, mx_m+56, my_m + 13 + i*12, 0)

    # --- MODALER BESTAETIGUNGSDIALOG (ueber Menue + Fenstern) ---
    draw_confirm()

    # --- DRAG-INDIKATOR + CURSOR (immer obenauf) ---
    if dragged_file:
        base = os.path.basename(dragged_file)[:15]
        tw = len(base) * 4 + 12
        py16.blend_mode("alpha", alpha=180)
        py16.rectfill(mx + 6, my + 6, tw, 12, 1); py16.blend_mode("normal"); py16.rect(mx
        py16.rect(mx + 8, my + 8, 6, 8, 7); py16.line(mx + 8, my + 8, mx + 10, my + 8, 7);

    py16.line(mx, my, mx+5, my+5, 0); py16.line(mx, my, mx, my+7, 0); py16.line(mx, my+7,
    py16.pset(mx+1, my+2, cursor_color); py16.line(mx+1, my+3, mx+2, my+3, cursor_color);

    # --- CRT MONITOR EFFEKT ---
    if crt_effect:
        py16.scanline_apply(py16.scanline_interlace(odd_offset=1, even_offset=-1), wrap=Fa

if __name__ == "__main__":
    py16.run(update, draw, init)
```